

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng ứng dụng hỗ trợ tư vấn bất động sản tích hợp Chatbot

TRẦN TRUNG HIẾU

Email: hieu.tt227114@sis.hust.edu.vn

Mã sinh viên: 20227114

Chuyên ngành Toán - Tin

Giảng viên hướng dẫn:

TS. Trần Anh Tú

Chữ ký của GVHD

HÀ NỘI, 6/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng ứng dụng hỗ trợ tư vấn bất động sản tích hợp Chatbot

TRẦN TRUNG HIẾU

Email: hieu.tt227114@sis.hust.edu.vn

Mã sinh viên: 20227114

Chuyên ngành Toán - Tin

Giảng viên hướng dẫn:

TS. Trần Anh Tú

Chữ ký của GVHD

HÀ NỘI, 6/2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Biểu mẫu của Đề tài/khóa luận tốt nghiệp theo qui định của viện, tuy nhiên cần đảm bảo giáo viên giao đề tài ký và ghi rõ họ và tên.

Trường hợp có 2 giáo viên hướng dẫn thì sẽ cùng ký tên.

Hà Nội, ngày tháng ...
năm ...

Giáo viên hướng dẫn

Ký và ghi rõ họ tên

PHIẾU BÁO CÁO TIẾN ĐỘ ĐỒ ÁN

- Phiếu này in từ hệ thống số hóa

Lời cảm ơn

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến TS. Trần Anh Tú , người đã tận tình hướng dẫn, định hướng và động viên em trong suốt quá trình thực hiện đồ án tốt nghiệp này. Những góp ý quý báu của thầy về kiến trúc hệ thống, phương pháp nghiên cứu và cách trình bày đã giúp em hoàn thiện đồ án một cách tốt nhất.

Em cũng xin gửi lời cảm ơn đến các thầy cô trong Khoa Toán – Tin, Đại học Bách Khoa Hà Nội, những người đã trang bị cho em nền tảng kiến thức vững chắc về công nghệ thông tin, trí tuệ nhân tạo và kỹ năng nghiên cứu trong suốt những năm học vừa qua.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, tạo điều kiện thuận lợi và là nguồn động viên tinh thần to lớn để em có thể tập trung hoàn thành đồ án này.

Tóm tắt nội dung đồ án

Đồ án này trình bày quá trình phân tích, thiết kế và xây dựng một nền tảng bất động sản thông minh tích hợp chatbot đa tác tử sử dụng kỹ thuật tạo sinh tăng cường truy xuất (RAG). Hệ thống hướng đến thị trường bất động sản Việt Nam, cung cấp cho người dùng khả năng duyệt và lọc danh sách nhà đất, xem chi tiết bất động sản kèm bản đồ, theo dõi thống kê thị trường, và đặc biệt là tương tác với chatbot đa tác tử có khả năng tư vấn cùng lúc nhiều lĩnh vực: tìm kiếm bất động sản, phân tích thị trường, tư vấn pháp lý và tư vấn đầu tư – tất cả thông qua giao diện hội thoại tự nhiên bằng tiếng Việt.

Về phương pháp thực hiện, đồ án áp dụng kiến trúc dịch vụ vi mô với sáu dịch vụ độc lập được điều phối qua Docker Compose: PostgreSQL tích hợp pgvector với chỉ mục HNSW cho tìm kiếm véc-tơ, Redis cho bộ nhớ đệm, Backend API xây dựng trên FastAPI với SQLAlchemy bất đồng bộ, Agent Service chạy đồ thị trạng thái LangGraph riêng biệt cho suy luận đa bước, Pipeline Worker đảm nhận thu thập và xử lý dữ liệu, cùng giao diện người dùng Next.js 16 với kết xuất phía máy chủ. Hệ thống tìm kiếm được thiết kế theo mô hình lai, kết hợp truy vấn có cấu trúc (lọc theo loại hình, khu vực, giá, diện tích, số phòng, hướng nhà) với tìm kiếm ngữ nghĩa qua véc-tơ đặc trưng 1024 chiều từ mô hình BGE-M3, có xếp hạng lại qua mô hình chuyên dụng đa ngôn ngữ. Chatbot sử dụng đồ thị trạng thái tám nút của LangGraph để điều phối luồng xử lý: xây dựng ngữ cảnh, kiểm tra trạng thái sẵn sàng dữ liệu, định tuyến ý định (dựa trên từ khóa hoặc mô hình ngôn ngữ lớn Gemini 2.0 Flash), lập kế hoạch truy xuất, thực thi sáu tác tử chuyên biệt song song (tìm kiếm bất động sản, phân tích thị trường, tư vấn đầu tư, tra cứu tin tức, tra cứu dự án, tư vấn pháp lý), tổng hợp kết quả có trích dẫn nguồn, kiểm tra an toàn (ràng buộc khuyến cáo pháp lý và tài

chính), và đề xuất ghi nhớ người dùng để cá nhân hóa trải nghiệm. Hệ thống còn tích hợp cơ chế đánh giá chất lượng phản hồi qua năm tiêu chí (tính căn cứ, tính hữu ích, chất lượng trích dẫn, an toàn, tính đầy đủ của vết xử lý), mô hình phân tích đầu tư với các chỉ số tài chính, và cơ chế ReAct cho suy luận lặp khi cần.

Dữ liệu được thu thập tự động từ batdongsan.com.vn qua công cụ tự động hóa trình duyệt với cơ chế ẩn danh, bao gồm bốn loại nội dung: tin bán, tin cho thuê, dự án và tin tức. Đường ống xử lý dữ liệu gồm đầy đủ các bước: làm sạch và chuẩn hóa (phân tích giá, diện tích, phân loại), làm giàu (mã hóa địa lý, gán nhãn nhu cầu), chia đoạn ngữ nghĩa, tạo véc-tơ đặc trưng và nạp vào cơ sở dữ liệu. Hệ thống còn có kho kiến thức pháp lý được xử lý từ tài liệu HTML và PDF. Toàn bộ đường ống được lập lịch tự động qua Apache Airflow với bốn luồng công việc định kỳ. Hệ thống giám sát toàn diện bao gồm theo dõi vết xử lý tác tử, chỉ số hiệu năng qua Prometheus và Grafana, cùng bộ kiểm thử hơn 55 tệp bao phủ toàn bộ các mô-đun.

Kết quả đạt được là một hệ thống hoàn chỉnh, có tính thực tế cao, sẵn sàng triển khai làm nền tảng dịch vụ bất động sản hoặc phát triển mở rộng theo hướng tích hợp thêm nguồn dữ liệu, cải thiện chất lượng véc-tơ đặc trưng và bổ sung tác tử chuyên biệt. Qua đồ án, em đã đạt được kiến thức và kỹ năng về thiết kế kiến trúc hệ thống toàn diện, xây dựng đường ống dữ liệu tự động, tích hợp cơ sở dữ liệu véc-tơ cho tìm kiếm ngữ nghĩa, phát triển hệ thống đa tác tử với LangGraph, và triển khai ứng dụng với Docker trên hạ tầng dịch vụ vi mô.

Hà Nội, ngày tháng ...
năm...

Sinh viên thực hiện

Ký và ghi rõ họ tên

Mục lục

GIỚI THIỆU	1
Đặt vấn đề	1
Mục tiêu của đề tài	2
Phạm vi nghiên cứu	4
Phương pháp tiếp cận	5
Bố cục của báo cáo	7
1 CƠ SỞ LÝ THUYẾT	9
1.1 Tổng quan về lĩnh vực nghiên cứu	9
1.1.1 Nền tảng bất động sản trực tuyến	9
1.1.2 Chatbot và trợ lý ảo trong bất động sản	10
1.2 Các khái niệm cơ bản	10
1.2.1 Kỹ thuật tạo sinh tăng cường truy xuất (RAG)	10
1.2.2 Hệ đa tác tử với LangGraph	11
1.2.3 Mẫu kiến trúc Bảng đen (Blackboard Pattern)	13
1.2.4 Mẫu kiến trúc Đánh giá Hội đồng (Committee Review)	14
1.2.5 Mẫu Suy luận – Hành động (ReAct)	15
1.2.6 Véc-tơ đặc trưng và Tìm kiếm ngữ nghĩa	15
1.2.7 Tìm kiếm lai và Xếp hạng lại	16
1.3 Các công nghệ sử dụng	17
1.3.1 FastAPI	17
1.3.2 Next.js 16	17
1.3.3 PostgreSQL + pgvector	18
1.3.4 LangGraph	18
1.3.5 Google Gemini 2.0 Flash	19
1.3.6 BGE-M3 Embedding	20
1.3.7 Cohere Rerank	20
1.3.8 Redis 7	21
1.3.9 Apache Airflow	21
1.3.10 Playwright	21
1.3.11 Docker & Docker Compose	22
1.3.12 Các thư viện và công cụ hỗ trợ	22

2	PHÂN TÍCH THIẾT KẾ HỆ THỐNG	24
2.1	Phân tích yêu cầu	24
2.1.1	Yêu cầu chức năng	24
2.1.2	Yêu cầu phi chức năng	30
2.2	Thiết kế kiến trúc hệ thống	32
2.2.1	Kiến trúc tổng thể	32
2.3	Thiết kế cơ sở dữ liệu	43
2.3.1	Mô hình quan hệ	43
2.3.2	Chiến lược indexing	44
2.4	Thiết kế giao diện người dùng	45
3	TRIỂN KHAI VÀ KIỂM THỬ	46
3.1	Triển khai hệ thống với Docker	46
3.1.1	Kiến trúc triển khai	46
3.1.2	Cấu hình Docker Compose	46
3.1.3	Cấu hình chi tiết từng dịch vụ	47
3.1.4	Mạng nội bộ và giao tiếp giữa các dịch vụ	51
3.1.5	Quy trình triển khai	51
3.2	Hệ thống giám sát và quan sát	54
3.2.1	Observability – Theo dõi vết xử lý tác tử	54
3.2.2	Đánh giá chất lượng tự động (LLM Judge)	54
3.2.3	Theo dõi chi phí LLM	55
3.2.4	Prometheus + Grafana + Alertmanager	55
3.2.5	Admin Dashboard	56
3.3	Kiểm thử hệ thống	56
3.3.1	Chiến lược kiểm thử	56
3.3.2	Công cụ và khung công tác	57
3.3.3	Kiểm thử chức năng theo phân tích yêu cầu	57
3.3.4	Kiểm thử hiệu năng	63
3.3.5	Kết quả kiểm thử	65
3.3.6	Các vấn đề đã phát hiện và hướng khắc phục	66
	KẾT LUẬN	67
	Kết quả đạt được	67
	Hạn chế	68
	Hướng phát triển	69

PHỤ LỤC	70
CHỈ MỤC	82
TÀI LIỆU THAM KHẢO	83

Danh sách hình vẽ

2.1	Kiến trúc tổng thể hệ thống Nền tảng Tư vấn Bất động sản Tích hợp Chatbot	33
2.2	Tổng quan 14 nút trong đồ thị trạng thái LangGraph của Agent Service . .	36
3.1	Kiến trúc triển khai Docker Compose với 6 dịch vụ	46
3.2	Sơ đồ kiến trúc tổng thể hệ thống (5 tầng: Người dùng – Frontend – Backend & Agent Service – Data – Pipeline)	75

Danh sách bảng

2.1	Các yêu cầu phi chức năng của hệ thống	32
3.1	Các ca kiểm thử chính theo nhóm chức năng	63
3.2	Chỉ tiêu hiệu năng dự kiến	65
3.3	Danh sách API endpoints chính của hệ thống (tiền tố mặc định: /api/v1)	79

DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT

Ký hiệu / Từ viết tắt	Diễn giải
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface
BGE-M3	BAAI General Embedding – Multilingual, Multi-functionality, Multi-granularity
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
ETL	Extract, Transform, Load
HNSW	Hierarchical Navigable Small World (thuật toán index vector)
JWT	JSON Web Token
kNN	k-Nearest Neighbors
LLM	Large Language Model – Mô hình ngôn ngữ lớn
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
ROI	Return on Investment
SQL	Structured Query Language
SSE	Server-Sent Events
SSR	Server-Side Rendering
VND	Việt Nam Đồng

GIỚI THIỆU ĐỀ TÀI

Đặt vấn đề

Thị trường bất động sản Việt Nam những năm gần đây chứng kiến sự phát triển sôi động với hàng trăm nghìn giao dịch mỗi năm và nhu cầu thông tin ngày càng cao từ phía người mua, người bán cũng như nhà đầu tư. Các cổng thông tin bất động sản trực tuyến như batdongsan.com.vn, alanhadat.com.vn đã trở thành kênh tiếp cận chính, với hàng chục nghìn tin đăng mới mỗi ngày. Tuy nhiên, việc khai thác thông tin từ các nền tảng này một cách hiệu quả vẫn còn nhiều thách thức.

Thách thức thứ nhất là tình trạng thông tin phân mảnh và quá tải. Dữ liệu bất động sản nằm rải rác trên nhiều nền tảng với hàng chục nghìn tin đăng. Người dùng phải tự duyệt qua hàng trăm kết quả, so sánh thủ công giữa các tin để tìm được bất động sản phù hợp. Việc tổng hợp thông tin từ nhiều nguồn và đưa ra quyết định dựa trên dữ liệu là một quá trình tốn nhiều thời gian và công sức. Các bộ lọc hiện có trên các nền tảng thường chỉ giới hạn ở loại hình, khu vực và khoảng giá cơ bản, chưa hỗ trợ các tiêu chí tinh tế hơn như hướng nhà, số phòng ngủ, số phòng tắm hay mặt tiền.

Thách thức thứ hai là sự thiếu hụt tư vấn chuyên sâu đa lĩnh vực. Một giao dịch bất động sản không chỉ đơn thuần là tìm kiếm và so sánh giá. Người mua cần được tư vấn về tình trạng pháp lý (sổ đỏ, quy hoạch, thủ tục sang tên, thuế và phí), xu hướng thị trường (biến động giá theo khu vực, loại hình bất động sản tiềm năng, thống kê giá trung bình) và tiềm năng đầu tư (khả năng sinh lời, tỷ suất vốn hóa, dòng tiền cho thuê, thanh khoản). Mỗi lĩnh vực đòi hỏi kiến thức chuyên môn sâu và khả năng tổng hợp thông tin từ nhiều nguồn khác nhau. Tuy nhiên, các nền tảng hiện tại chủ yếu chỉ hiển thị danh sách tin đăng và bộ lọc cơ bản, chưa có khả năng tư vấn tổng hợp đa chiều.

Thách thức thứ ba là rào cản ngôn ngữ trong tìm kiếm ngữ nghĩa. Các giải pháp tìm kiếm truyền thống dựa trên khớp từ khóa chính xác (LIKE trong cơ sở dữ liệu quan hệ) không hiểu được ngữ nghĩa và ngữ cảnh của truy vấn tiếng Việt. Ví dụ, người dùng tìm “căn hộ thoáng mát gần công viên, view sông, dưới 3 tỷ” yêu cầu hệ thống phải hiểu được các khái niệm trừu tượng như “thoáng mát”, “gần công viên”, “view sông” – những khái niệm không thể tra cứu bằng câu lệnh truy vấn thông thường. Tiếng Việt với đặc thù từ đa âm tiết, thanh điệu và từ đồng nghĩa phong phú càng làm tăng độ phức tạp cho bài toán tìm kiếm ngữ nghĩa.

Thách thức thứ tư là thiếu tự động hóa trong thu thập và xử lý dữ liệu. Quy trình thu thập dữ liệu bất động sản thường thủ công, không theo kịp tốc độ cập nhật của thị

trường. Dữ liệu thô từ các nền tảng cần được làm sạch (chuẩn hóa định dạng giá, diện tích, số phòng; phân loại loại hình và nhu cầu), làm giàu (gán tọa độ địa lý cho phép tìm kiếm theo không gian; trích xuất đặc trưng từ mô tả), chia đoạn ngữ nghĩa (tách thông tin thành các đơn vị có ý nghĩa để truy xuất chính xác) và cập nhật định kỳ mới có thể sử dụng được cho các hệ thống tìm kiếm thông minh. Nếu không có đường ống tự động hóa, việc duy trì cơ sở dữ liệu luôn mới và sạch là bất khả thi ở quy mô hàng chục nghìn tin đăng mỗi ngày.

Thách thức thứ năm là thiếu công cụ phân tích thị trường trực quan và tương tác. Người dùng phổ thông và nhà đầu tư thiếu các công cụ giúp họ nắm bắt bức tranh tổng quan về thị trường như giá trung bình theo khu vực, phân bố loại hình bất động sản, số lượng tin đăng theo thời gian, và xu hướng biến động giá. Các số liệu thống kê thường nằm trong các báo cáo định kỳ dạng PDF, không có sẵn dưới dạng tương tác trực tuyến với biểu đồ và khả năng lọc theo thời gian thực.

Trước bối cảnh đó, việc xây dựng một nền tảng bất động sản thông minh, tích hợp trợ lý ảo có khả năng hiểu ngôn ngữ tự nhiên tiếng Việt và tư vấn đa lĩnh vực (từ tìm kiếm bất động sản, phân tích thị trường, tra cứu dự án và tin tức, tư vấn pháp lý đến đánh giá đầu tư) là một nhu cầu cấp thiết. Hệ thống cần được xây dựng trên nền tảng dữ liệu được thu thập và cập nhật tự động, đảm bảo thông tin luôn mới và đáng tin cậy, đồng thời cung cấp các công cụ phân tích trực quan giúp người dùng ra quyết định dựa trên dữ liệu.

Mục tiêu của đề tài

Đề án hướng đến việc xây dựng một hệ thống toàn diện giải quyết năm thách thức đã nêu, với năm mục tiêu chính sau đây.

Mục tiêu thứ nhất: Xây dựng nền tảng web bất động sản hoàn chỉnh. Nền tảng sử dụng công nghệ kết xuất phía máy chủ (SSR) cho hiệu năng cao và tối ưu hóa cho công cụ tìm kiếm, bao gồm: trang duyệt danh sách nhà đất bán và cho thuê với phân trang và sắp xếp đa tiêu chí (giá, diện tích, ngày đăng); bộ lọc nâng cao theo loại tin, loại hình bất động sản, thành phố, quận/huyện, khoảng giá, diện tích, số phòng ngủ, số phòng tắm, hướng nhà và từ khóa; trang chi tiết bất động sản với đầy đủ thông số kỹ thuật (giá, diện tích, pháp lý, nội thất, hướng, mặt tiền, số tầng, tọa độ bản đồ); bảng thống kê thị trường trực quan với các chỉ số tổng quan và biểu đồ phân tích; hệ thống xác thực người dùng qua mã thông báo JWT; và bảng điều khiển quản trị theo dõi trạng thái đường ống dữ liệu và vết xử lý của tác tử.

Mục tiêu thứ hai: Phát triển chatbot đa tác tử tích hợp kỹ thuật tạo sinh tăng cường truy xuất (RAG). Hệ thống chatbot được thiết kế theo kiến trúc hai lớp: đường ống nội bộ nhúng trong máy chủ backend xử lý nhanh truy vấn đơn giản, và Dịch vụ Tác tử

(Agent Service) chạy đồ thị trạng thái LangGraph riêng biệt cho suy luận đa bước nâng cao. Đồ thị trạng thái gồm tám nút xử lý: xây dựng ngữ cảnh, kiểm tra trạng thái sẵn sàng dữ liệu, định tuyến ý định, thấu hiểu truy vấn, lập kế hoạch truy xuất, thực thi sáu tác tử chuyên biệt song song (tìm kiếm bất động sản, tra cứu dự án, tra cứu tin tức, phân tích thị trường, tư vấn pháp lý, tư vấn đầu tư), tổng hợp kết quả và kiểm tra an toàn, cùng đề xuất ghi nhớ người dùng. Hệ thống hỗ trợ tiếng Việt toàn diện từ nhận diện ý định, trích xuất bộ lọc đến sinh phản hồi; tích hợp cơ chế ghi nhớ và sở thích người dùng để cá nhân hóa trải nghiệm; cơ chế đánh giá chất lượng phản hồi qua năm tiêu chí (tính căn cứ, tính hữu ích, chất lượng trích dẫn, an toàn, tính đầy đủ của vết xử lý); mô hình phân tích đầu tư với các chỉ số tài chính; và cơ chế ReAct cho suy luận lặp khi cần.

Mục tiêu thứ ba: Xây dựng đường ống thu thập và xử lý dữ liệu tự động. Hệ thống thu thập dữ liệu sử dụng công cụ tự động hóa trình duyệt (Playwright) với cơ chế ẩn danh để lấy dữ liệu từ `batdongsan.com.vn` cho bốn loại nội dung: tin bán, tin cho thuê, dự án và tin tức. Đường ống xử lý bao gồm đầy đủ các bước: làm sạch (phân tích và chuẩn hóa giá, diện tích, số phòng; phân loại loại hình và nhu cầu), làm giàu (mã hóa địa lý qua dịch vụ bản đồ, gán nhãn nhu cầu), chia đoạn ngữ nghĩa (tách mỗi bất động sản thành bốn loại đoạn: tổng quan, chi tiết, vị trí, tài chính), tạo véc-tơ đặc trưng 1024 chiều bằng mô hình BGE-M3 và nạp vào PostgreSQL tích hợp pgvector qua các mô-đun chuyên biệt. Hệ thống còn có kho kiến thức pháp lý với khả năng xử lý tài liệu HTML và PDF. Toàn bộ đường ống được lập lịch tự động qua Apache Airflow với bốn luồng công việc: hàng ngày cho tin đăng, hàng tuần cho dự án và tin tức, hàng tháng cho kiến thức pháp lý.

Mục tiêu thứ tư: Triển khai hệ thống tìm kiếm lai hiệu quả. Hệ thống kết hợp tìm kiếm theo bộ lọc có cấu trúc (SQL trên các trường quan hệ: loại hình, khu vực, giá, diện tích, số phòng, hướng nhà) với tìm kiếm ngữ nghĩa qua chỉ mục HNSW trên PostgreSQL với pgvector. Hệ thống tích hợp xếp hạng lại qua mô hình chuyên dụng đa ngôn ngữ để cải thiện độ chính xác của kết quả, bộ nhớ đệm Redis cho cả véc-tơ đặc trưng và kết quả tìm kiếm nhằm giảm độ trễ, cùng khả năng trích xuất bộ lọc tự động từ truy vấn tiếng Việt bằng ngôn ngữ tự nhiên.

Mục tiêu thứ năm: Thiết kế kiến trúc dịch vụ vi mô có khả năng mở rộng và giám sát toàn diện. Hệ thống gồm sáu dịch vụ độc lập được điều phối qua Docker Compose: PostgreSQL tích hợp pgvector (cơ sở dữ liệu), Redis (bộ nhớ đệm), Backend API (máy chủ FastAPI với SQLAlchemy bất đồng bộ), Agent Service (máy chủ FastAPI với đồ thị trạng thái LangGraph), Pipeline Worker (thu thập và xử lý dữ liệu), và Frontend (giao diện Next.js 16 với kết xuất phía máy chủ). Hệ thống giám sát toàn diện bao gồm: theo dõi vết xử lý tác tử qua từng bước (thời gian, trạng thái, đầu ra); chỉ số hiệu năng qua Prometheus và

Grafana; cảnh báo qua Alertmanager; và thu thập phản hồi đánh giá chất lượng từ người dùng. Bộ kiểm thử hơn 55 tập bao phủ toàn bộ các mô-đun chính của hệ thống.

Phạm vi nghiên cứu

Đồ án tập trung vào thị trường bất động sản Việt Nam. Dữ liệu được thu thập từ nguồn chính là `batdongsan.com.vn` – cổng thông tin bất động sản lớn nhất Việt Nam với hàng chục nghìn tin đăng mới mỗi ngày, bao phủ tất cả 63 tỉnh thành trên cả nước.

Về loại hình bất động sản: Hệ thống bao gồm cả tin bán và tin cho thuê, với các loại hình chính: căn hộ chung cư, nhà riêng, nhà mặt phố, đất nền, biệt thự, nhà phố thương mại (shophouse) và đất dự án. Ngoài ra, hệ thống còn thu thập và lập chỉ mục dữ liệu về các dự án bất động sản (thông tin chủ đầu tư, quy mô, tiến độ, vị trí) và tin tức thị trường (biến động giá, chính sách mới, xu hướng đầu tư).

Về chức năng: Hệ thống phục vụ ba nhóm đối tượng chính. *Đối với người dùng cuối:* duyệt và lọc danh sách bất động sản với bộ lọc nâng cao (loại tin, loại hình, thành phố, quận/huyện, khoảng giá, diện tích, số phòng ngủ, số phòng tắm, hướng nhà, từ khóa); xem chi tiết bất động sản với ba mươi trường thông tin và bản đồ tương tác; tương tác với chatbot đa tác tử có khả năng tư vấn đồng thời sáu lĩnh vực (tìm kiếm bất động sản, tra cứu dự án, tra cứu tin tức, phân tích thị trường, tư vấn pháp lý, tư vấn đầu tư); xem thống kê thị trường trực quan với biểu đồ tương tác; đăng ký và xác thực tài khoản qua mã thông báo JWT; và quản lý sở thích cá nhân để cá nhân hóa trải nghiệm. *Đối với quản trị viên:* giám sát trạng thái đường ống dữ liệu qua bảng điều khiển; theo dõi vết xử lý của tác tử (từng bước, thời gian, trạng thái, đầu ra); quản lý phản hồi người dùng và đề xuất ghi nhớ; theo dõi chi phí sử dụng mô hình ngôn ngữ lớn. *Đối với hệ thống:* thu thập dữ liệu tự động theo lịch từ `batdongsan.com.vn` cho bốn loại nội dung; xử lý toàn diện từ làm sạch đến tạo véc-tơ đặc trưng; tạo và lập chỉ mục HNSW trên PostgreSQL với pgvector; kiểm tra trạng thái sẵn sàng của các nguồn dữ liệu trước khi truy xuất; giám sát hiệu năng và cảnh báo qua Prometheus, Grafana và Alertmanager.

Về công nghệ: Đồ án sử dụng các công nghệ hiện đại và được lựa chọn phù hợp với từng tầng của hệ thống. Tầng giao diện người dùng: Next.js 16 + React 19 (TypeScript) với Bộ định tuyến ứng dụng (App Router), kết xuất phía máy chủ (SSR) và Tailwind CSS phiên bản 4; thư viện biểu đồ Recharts cho trực quan hóa dữ liệu. Tầng máy chủ dịch vụ: FastAPI (Python 3.12), SQLAlchemy 2.0 bất đồng bộ, Pydantic phiên bản 2 cho xác thực dữ liệu; Alembic cho di chuyển cơ sở dữ liệu; bcrypt và PyJWT cho xác thực người dùng. Tầng Dịch vụ Tác tử: FastAPI + LangGraph với đồ thị trạng thái tám nút, Google Gemini 2.0 Flash cho mô hình ngôn ngữ chính và mô hình đánh giá, hỗ trợ tùy chọn mô hình Cohere

cho xếp hạng lại. Tầng dữ liệu: PostgreSQL 16 tích hợp pgvector với chỉ mục HNSW cho tìm kiếm véc-tơ; Redis 7 cho bộ nhớ đệm véc-tơ đặc trưng và kết quả truy vấn. Tầng xử lý ngôn ngữ và tìm kiếm: BGE-M3 (BAAI) cho véc-tơ đặc trưng 1024 chiều tối ưu cho tiếng Việt; mô hình Xếp hạng lại Đa ngôn ngữ cho sắp xếp kết quả; Google Gemini 2.0 Flash cho phân loại ý định và sinh phản hồi tiếng Việt. Tầng thu thập và xử lý dữ liệu: Playwright với cơ chế ẩn danh cho thu thập dữ liệu; Apache Airflow cho lập lịch đường ống; công cụ làm sạch và làm giàu dữ liệu tùy chỉnh; dịch vụ mã hóa địa lý cho gán tọa độ. Tầng hạ tầng và triển khai: Docker Compose cho điều phối dịch vụ; Prometheus và Grafana cho giám sát; Alertmanager cho cảnh báo.

Về ngôn ngữ: Giao diện người dùng, phản hồi chatbot, văn bản hệ thống và tài liệu hướng dẫn sử dụng tiếng Việt. Mã nguồn, chú thích trong mã, tên biến, tên hàm và tên lớp sử dụng tiếng Anh theo quy ước quốc tế. Đường dẫn thân thiện với người dùng và công cụ tìm kiếm sử dụng tiếng Việt không dấu (/nha-dat-ban, /nha-dat-cho-thue, /thi-truong, /tin-tuc, /du-an, /dang-nhap, /dang-ky).

Về giới hạn: Đồ án không bao gồm: xử lý giao dịch thực tế (đặt cọc, thanh toán, ký hợp đồng điện tử); tích hợp với hệ thống ngân hàng hoặc cổng thanh toán; ứng dụng di động native (iOS/Android); xử lý dữ liệu thời gian thực dạng luồng (streaming); và huấn luyện hoặc tinh chỉnh mô hình ngôn ngữ lớn – hệ thống chỉ sử dụng mô hình có sẵn thông qua API.

Phương pháp tiếp cận

Đồ án được thực hiện theo hướng tiếp cận phát triển theo mô-đun từ dưới lên (bottom-up), kết hợp giữa nghiên cứu lý thuyết và triển khai thực nghiệm. Quy trình thực hiện gồm sáu giai đoạn chính, mỗi giai đoạn giải quyết một khía cạnh của hệ thống và được kiểm thử độc lập trước khi tích hợp.

Giai đoạn 1 – Nghiên cứu nền tảng lý thuyết và khảo sát công nghệ. Giai đoạn đầu tiên tập trung vào nghiên cứu các khái niệm nền tảng: kỹ thuật tạo sinh tăng cường truy xuất (RAG), kiến trúc hệ đa tác tử, véc-tơ đặc trưng và tìm kiếm ngữ nghĩa, tìm kiếm lai và xếp hạng lại. Song song đó, đồ án tiến hành khảo sát và đánh giá các công nghệ hiện có để lựa chọn bộ công cụ phù hợp: LangGraph cho điều phối đa tác tử (so với CrewAI, AutoGen), PostgreSQL với pgvector cho tìm kiếm véc-tơ (so với ChromaDB, Pinecone, Qdrant), BGE-M3 cho véc-tơ đặc trưng tiếng Việt (so với E5, BGE-M3-embedding), và Google Gemini 2.0 Flash cho mô hình ngôn ngữ (so với GPT-4o, Claude 3.5). Tiêu chí lựa chọn dựa trên: khả năng hỗ trợ tiếng Việt, chi phí vận hành, khả năng mở rộng và tính nguồn mở.

Giai đoạn 2 – Thu thập và xây dựng đường ống dữ liệu. Đây là giai đoạn nền tảng, được thực hiện đầu tiên vì toàn bộ hệ thống phụ thuộc vào chất lượng dữ liệu. Đồ án xây dựng công cụ thu thập dữ liệu sử dụng Playwright với cơ chế ẩn danh (stealth) để trích xuất dữ liệu từ batdongsan.com.vn cho bốn loại nội dung: tin bán, tin cho thuê, dự án và tin tức. Dữ liệu thô sau đó được đưa qua đường ống xử lý ETL gồm: làm sạch (chuẩn hóa giá, diện tích, số phòng; loại bỏ trùng lặp; phân loại loại hình và nhu cầu), làm giàu (mã hóa địa lý qua dịch vụ bản đồ; gán nhãn nhu cầu mua/thuê; trích xuất đặc trưng từ tiêu đề và mô tả), chia đoạn ngữ nghĩa (mỗi bất động sản được tách thành bốn loại đoạn: tổng quan, mô tả chi tiết, thông tin vị trí, thông tin tài chính), và tạo véc-tơ đặc trưng 1024 chiều bằng mô hình BGE-M3. Đường ống được đóng gói trong dịch vụ Pipeline Worker riêng biệt và lập lịch tự động qua Apache Airflow. Kho kiến thức pháp lý được xây dựng riêng từ các văn bản pháp luật định dạng HTML và PDF, xử lý qua cùng đường ống chia đoạn và tạo véc-tơ.

Giai đoạn 3 – Xây dựng nền tảng web và máy chủ dịch vụ. Giai đoạn này phát triển giao diện người dùng và máy chủ API. Giao diện người dùng được xây dựng với Next.js 16 sử dụng Bộ định tuyến ứng dụng, kết xuất phía máy chủ (SSR) để tối ưu hiệu năng và SEO, và Tailwind CSS phiên bản 4 cho thiết kế giao diện. Các trang được phát triển tuần tự: trang chủ, duyệt danh sách nhà đất bán và cho thuê, chi tiết bất động sản, thống kê thị trường, đăng nhập/đăng ký, và bảng điều khiển quản trị. Máy chủ dịch vụ được xây dựng với FastAPI theo kiến trúc phân lớp: lớp định tuyến (routers) xử lý yêu cầu HTTP, lớp dịch vụ (services) chứa logic nghiệp vụ, lớp mô hình (models) định nghĩa ORM với SQLAlchemy bất đồng bộ, và lớp giản đồ (schemas) xác thực dữ liệu với Pydantic phiên bản 2. Hệ thống tìm kiếm lai được tích hợp ở tầng dịch vụ, kết hợp truy vấn SQL có cấu trúc với tìm kiếm véc-tơ qua chỉ mục HNSW.

Giai đoạn 4 – Phát triển chatbot đa tác tử. Chatbot được phát triển theo kiến trúc hai lớp. Lớp thứ nhất là đường ống nội bộ nhúng trong máy chủ backend, sử dụng đồ thị LangGraph đơn giản gồm ba bước: định tuyến ý định, thực thi tác tử, tổng hợp phản hồi. Lớp thứ hai là Dịch vụ Tác tử (Agent Service) chạy độc lập, sử dụng đồ thị trạng thái LangGraph mười bốn nút với quy trình xử lý đầy đủ qua bốn pha: Hiểu (xây dựng ngữ cảnh, kiểm tra trạng thái sẵn sàng dữ liệu, định tuyến ý định với ba chế độ: từ khóa/mô hình ngôn ngữ/kết hợp, làm rõ truy vấn), Truy xuất (thấu hiểu truy vấn, lập kế hoạch và thực thi truy xuất đa nguồn), Suy luận (thực thi sáu tác tử chuyên biệt song song, mô hình phân tích đầu tư, đánh giá hội đồng đa góc nhìn), và Phản hồi (tổng hợp kết quả có trích dẫn nguồn, kiểm tra an toàn, điều khiển phản ứng ReAct, đề xuất ghi nhớ người dùng). Các cơ chế nâng cao được phát triển theo hướng tùy chọn (có thể bật/tắt qua biến môi trường): mô hình phân tích đầu tư với các chỉ số tài chính (NPV, IRR, tỷ suất vốn hóa, dòng tiền), cơ chế ReAct cho suy

luyện tập, đánh giá chất lượng phản hồi qua năm tiêu chí, và cơ chế ghi nhớ người dùng. Việc phát triển theo hướng tùy chọn cho phép hệ thống hoạt động ổn định ở chế độ cơ bản trong khi các tính năng nâng cao được thử nghiệm và hoàn thiện dần.

Giai đoạn 5 – Tích hợp, kiểm thử và đánh giá. Sau khi các mô-đun được phát triển độc lập, đồ án tiến hành tích hợp toàn bộ hệ thống qua Docker Compose với sáu dịch vụ. Chiến lược kiểm thử đa cấp độ được áp dụng: kiểm thử đơn vị cho từng hàm và lớp (hơn 60 tệp kiểm thử), kiểm thử tích hợp cho các tương tác giữa các mô-đun (cơ sở dữ liệu, Redis, dịch vụ tác tử, Pipeline Worker), và kiểm thử chấp nhận cho các luồng người dùng chính (duyet danh sách, lọc, chatbot, đăng nhập). Hệ thống đánh giá chất lượng chatbot được triển khai với năm tiêu chí: tính căn cứ (phản hồi có được hỗ trợ bởi nguồn dữ liệu không), tính hữu ích (phản hồi có trực tiếp giúp ích cho người dùng không), chất lượng trích dẫn (nguồn tham khảo có liên quan không), an toàn (có khuyến cáo pháp lý và tài chính phù hợp không), và tính đầy đủ của vết xử lý (các bước xử lý có được ghi nhận đầy đủ không).

Giai đoạn 6 – Triển khai và giám sát. Hệ thống được đóng gói và triển khai qua Docker Compose với cấu hình kiểm tra trạng thái (healthcheck) và chuỗi phụ thuộc giữa các dịch vụ. Hệ thống giám sát toàn diện được thiết lập: Prometheus thu thập chỉ số hiệu năng từ Backend API và Agent Service, Grafana trực quan hóa các chỉ số qua bảng điều khiển, Alertmanager gửi cảnh báo khi vượt ngưỡng. Ngoài ra, hệ thống còn theo dõi vết xử lý chi tiết của từng bước trong đồ thị tác tử (thời gian, trạng thái, đầu ra) và chi phí sử dụng mô hình ngôn ngữ lớn để tối ưu chi phí vận hành.

Bố cục của báo cáo

Báo cáo được tổ chức thành ba chương chính, phần Giới thiệu và phần Kết luận.

Phần **Giới thiệu** trình bày vấn đề về những thách thức trong tiếp cận thông tin bất động sản tại Việt Nam; xác định năm mục tiêu chính của đồ án; phạm vi nghiên cứu về thị trường, loại hình, chức năng và công nghệ; phương pháp tiếp cận theo hướng phát triển mô-đun; và những đóng góp chính của đồ án, trong đó nhấn mạnh quy trình thu thập dữ liệu, phương pháp chia đoạn ngữ nghĩa và cách thức xây dựng hệ thống tạo sinh tăng cường truy xuất.

Chương 1 – Cơ sở lý thuyết trình bày tổng quan về lĩnh vực nghiên cứu gồm nền tảng bất động sản trực tuyến và ứng dụng chatbot trong lĩnh vực này. Tiếp theo là các khái niệm nền tảng: kỹ thuật tạo sinh tăng cường truy xuất, hệ đa tác tử, véc-tơ đặc trưng và tìm kiếm ngữ nghĩa, tìm kiếm lai và xếp hạng lại. Phần cuối chương giới thiệu chi tiết từng công nghệ được sử dụng: FastAPI, Next.js, PostgreSQL với tìm kiếm véc-tơ, LangGraph, Google Gemini, mô hình véc-tơ BGE-M3, mô hình xếp hạng lại, Redis, Apache Airflow

và công cụ tự động hóa trình duyệt.

Chương 2 – Phân tích thiết kế hệ thống trình bày toàn bộ quá trình phân tích yêu cầu chức năng (ba nhóm: người dùng cuối, chatbot, quản trị và dữ liệu) và phi chức năng. Tiếp đến là thiết kế kiến trúc tổng thể năm tầng với sơ đồ kiến trúc; thiết kế chi tiết các tầng (giao diện người dùng, máy chủ dịch vụ, dịch vụ tác tử, tầng dữ liệu, đường ống dữ liệu); thiết kế cơ sở dữ liệu với bốn nhóm bảng và chiến lược lập chỉ mục; cùng thiết kế giao diện người dùng cho sáu loại trang chính.

Chương 3 – Triển khai và kiểm thử trình bày chi tiết quy trình triển khai hệ thống với Docker Compose (năm dịch vụ, kiểm tra trạng thái, chuỗi phụ thuộc); cấu hình môi trường và biến môi trường; quy trình tích hợp liên tục và giám sát (chỉ số hệ thống, theo dõi vết tác tử, cảnh báo); lập lịch đường ống với Apache Airflow (bốn luồng công việc); chiến lược kiểm thử đa cấp độ với 58 tệp kiểm thử; kết quả kiểm thử chi tiết theo mười nhóm chức năng; và đánh giá hiệu năng hệ thống.

Phần **Kết luận** tổng kết các kết quả chính đạt được, phân tích những hạn chế hiện tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

Chương này trình bày các khái niệm nền tảng và công nghệ được sử dụng trong đề án. Phần đầu giới thiệu tổng quan về lĩnh vực nghiên cứu: nền tảng bất động sản trực tuyến và ứng dụng của chatbot trong lĩnh vực này. Phần tiếp theo đi sâu vào các khái niệm cốt lõi: kỹ thuật tạo sinh tăng cường truy xuất (RAG), hệ đa tác tử với LangGraph, mẫu kiến trúc Bảng đen và Hội đồng, mẫu Suy luận – Hành động (ReAct), véc-tơ đặc trưng và tìm kiếm ngữ nghĩa, tìm kiếm lai và xếp hạng lại. Phần cuối cùng giới thiệu chi tiết từng công nghệ được sử dụng trong hệ thống, từ giao diện người dùng, máy chủ dịch vụ, cơ sở dữ liệu đến trí tuệ nhân tạo và đường ống dữ liệu.

1.1. Tổng quan về lĩnh vực nghiên cứu

1.1.1. Nền tảng bất động sản trực tuyến

Các nền tảng bất động sản trực tuyến đóng vai trò trung gian kết nối người mua, người bán và nhà đầu tư. Tại Việt Nam, batdongsan.com.vn là cổng thông tin bất động sản lớn nhất với hàng chục nghìn tin đăng mới mỗi ngày, bao gồm cả tin bán và cho thuê với đa dạng loại hình: căn hộ chung cư, nhà riêng, nhà mặt phố, đất nền, biệt thự và nhà phố thương mại (shophouse). Các nền tảng này thường cung cấp chức năng đăng tin, tìm kiếm theo bộ lọc cơ bản (giá, diện tích, khu vực) và hiển thị thông tin chi tiết của từng bất động sản.

Tuy nhiên, các nền tảng hiện tại còn tồn tại nhiều hạn chế:

- **Tìm kiếm dựa trên từ khóa chính xác:** Hệ thống chỉ khớp chính xác từ khóa người dùng nhập với nội dung tin đăng, không hiểu được ngữ nghĩa và ngữ cảnh của truy vấn. Ví dụ, truy vấn “căn hộ thoáng mát, view đẹp” không thể được xử lý hiệu quả bằng SQL WHERE thông thường.
- **Thiếu tư vấn tổng hợp đa lĩnh vực:** Người dùng phải tự tổng hợp thông tin từ nhiều nguồn khác nhau để đưa ra quyết định. Không có cơ chế tự động phân tích xu hướng thị trường, so sánh giá giữa các khu vực, đánh giá tiềm năng đầu tư, hay tra cứu thông tin pháp lý một cách có hệ thống.
- **Dữ liệu không được tổ chức cho AI:** Dữ liệu trên các nền tảng được thiết kế để hiển thị cho con người đọc, không được cấu trúc và đánh chỉ mục để phục vụ các hệ thống truy xuất thông tin (information retrieval) và sinh văn bản (text generation).

1.1.2. Chatbot và trợ lý ảo trong bất động sản

Chatbot trong lĩnh vực bất động sản có tiềm năng tự động hóa quy trình tư vấn, giảm tải cho nhân viên môi giới và cung cấp dịch vụ liên tục cho khách hàng. Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Models – LLMs) như GPT-4, Google Gemini, Claude đã mở ra khả năng xây dựng các chatbot thông minh, có khả năng hiểu ngôn ngữ tự nhiên và sinh phản hồi mạch lạc.

Tuy nhiên, LLM thuần túy có hai hạn chế lớn khi áp dụng vào lĩnh vực bất động sản. Thứ nhất là hiện tượng ảo giác (hallucination): mô hình có thể sinh ra thông tin không chính xác về giá cả, vị trí hoặc tình trạng pháp lý của bất động sản – những thông tin đòi hỏi độ chính xác tuyệt đối trong giao dịch thực tế. Thứ hai là thông tin lỗi thời: mô hình được huấn luyện trên dữ liệu có thời điểm cắt nhất định, không thể cập nhật theo biến động hàng ngày của thị trường.

Để khắc phục, các hệ thống chatbot hiện đại kết hợp LLM với kỹ thuật tạo sinh tăng cường truy xuất (RAG), cho phép mô hình truy xuất thông tin từ cơ sở dữ liệu ngoài trước khi sinh phản hồi. Cách tiếp cận này đảm bảo phản hồi dựa trên dữ liệu thực tế, có thể kiểm chứng và luôn được cập nhật. Hơn nữa, kiến trúc đa tác tử (multi-agent) cho phép phân chia bài toán tư vấn phức tạp thành các tác vụ chuyên biệt, mỗi tác vụ do một tác tử đảm nhận với kiến thức và công cụ riêng.

1.2. Các khái niệm cơ bản

1.2.1. Kỹ thuật tạo sinh tăng cường truy xuất (RAG)

Kỹ thuật tạo sinh tăng cường truy xuất (Retrieval-Augmented Generation – RAG) là kỹ thuật kết hợp giữa truy xuất thông tin và sinh văn bản, được đề xuất bởi Lewis và cộng sự (2020). Quy trình gồm ba bước chính: truy xuất, tăng cường và sinh.

1. **Truy xuất (Retrieve):** Khi nhận được câu hỏi của người dùng, hệ thống chuyển câu hỏi thành vector embedding và tìm kiếm k đoạn văn bản (chunks) có độ tương đồng cao nhất trong cơ sở dữ liệu vector. Phép đo tương đồng thường dùng là cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|} \quad (1.1)$$

trong đó $\mathbf{q}$ là vector embedding của câu truy vấn, $\mathbf{d}_i$ là vector embedding của chunk thứ i .

2. **Tăng cường (Augment):** Các chunks được truy xuất sẽ được đưa vào prompt của

LLM như ngữ cảnh bổ sung (context). Prompt thường có cấu trúc: system instruction + retrieved context + user query.

3. **Sinh (Generate):** LLM sinh câu trả lời dựa trên câu hỏi gốc và ngữ cảnh được cung cấp, kèm theo trích dẫn nguồn (citations) cho phép người dùng kiểm chứng thông tin.

Ưu điểm của RAG:

- **Giảm thiểu ảo giác:** Phản hồi được neo (grounded) vào dữ liệu thực tế thay vì chỉ dựa vào kiến thức nội tại của LLM.
- **Thông tin cập nhật:** Cơ sở dữ liệu vector có thể được cập nhật liên tục mà không cần fine-tune lại LLM.
- **Kiểm chứng được:** Mỗi phản hồi đi kèm danh sách nguồn tham khảo.
- **Domain-specific:** Có thể dễ dàng áp dụng cho lĩnh vực chuyên biệt bằng cách xây dựng kho dữ liệu riêng.

Các biến thể RAG trong đồ án:

- **Simple RAG:** Truy xuất đơn giản – query embedding → vector search → LLM generation. Được sử dụng làm fallback.
- **Agent-based RAG:** Mỗi agent chuyên biệt thực hiện truy xuất riêng cho domain của mình (property, market, legal, investment, news, project), sau đó kết quả được tổng hợp.
- **Multi-step RAG (Agent Service):** Sử dụng retrieval planner để lập kế hoạch truy xuất qua nhiều bước, mỗi bước có thể truy xuất từ các nguồn khác nhau, có cơ chế ReAct để truy xuất bổ sung khi thiếu bằng chứng.

1.2.2. Hệ đa tác tử với LangGraph

Khái niệm hệ đa tác tử

Hệ đa tác tử (Multi-Agent System – MAS) là kiến trúc trong đó nhiều tác tử chuyên biệt cùng phối hợp để giải quyết một nhiệm vụ phức tạp. Mỗi agent đảm nhận một vai trò riêng, có system prompt riêng, và có thể truy cập các công cụ (tools) khác nhau. Kiến trúc này đặc biệt phù hợp với bài toán tư vấn bất động sản vì một truy vấn của người dùng thường liên quan đến nhiều khía cạnh: tìm kiếm bất động sản, phân tích thị trường, tra cứu dự án, cập nhật tin tức, tư vấn pháp lý và đánh giá đầu tư.

Hệ thống trong đồ án triển khai kiến trúc hai lớp. Lớp thứ nhất là chatbot pipeline nội bộ trong Backend API với router (phân loại ý định) và orchestrator (điều phối agent), xử lý nhanh các truy vấn đơn giản. Lớp thứ hai là Agent Service – dịch vụ vi mô riêng biệt sử dụng LangGraph để điều phối quy trình đa tác tử phức tạp qua đồ thị trạng thái.

LangGraph và Đồ thị trạng thái

LangGraph là thư viện của LangChain cho phép xây dựng các ứng dụng đa tác tử có trạng thái (stateful) dưới dạng đồ thị có hướng (Directed Graph). Các khái niệm cốt lõi:

- **StateGraph:** Đồ thị trạng thái với một state object (TypedDict) được truyền qua tất cả các node. Mỗi node đọc và ghi vào state, cho phép tích lũy thông tin qua từng bước xử lý.
- **Nodes:** Các hàm xử lý Python thuần túy, nhận state và trả về dict cập nhật một phần state. Mỗi node đại diện cho một bước trong quy trình.
- **Edges:** Kết nối có hướng giữa các node. Hai loại: normal edge (luôn chuyển từ node A sang node B) và conditional edge (chuyển đến node khác nhau dựa trên giá trị trong state).
- **Compilation:** Đồ thị được biên dịch (compile) thành một executable graph có thể gọi qua `ainvoke()` bất đồng bộ.

Trong Agent Service của đồ án, đồ thị trạng thái gồm 14 nút được tổ chức thành bốn pha xử lý chính:

1. **Pha Hiểu (Understand):** `context_builder` (chuẩn hóa truy vấn, compact ngữ cảnh), `readiness_checker` (kiểm tra trạng thái sẵn sàng của 4 nguồn dữ liệu), `router` (phân loại ý định, chọn danh sách tác tử), `clarification` (tạo câu hỏi làm rõ nếu cần).
2. **Pha Truy xuất (Retrieve):** `query_understanding` (viết lại truy vấn, trích xuất bộ lọc có cấu trúc), `retrieval_planner` (lập kế hoạch và thực thi truy xuất đa nguồn).
3. **Pha Suy luận (Reason):** `specialist_agents` (thực thi song song sáu tác tử chuyên biệt), `investment_model` (xây dựng mô hình tài chính đầu tư), `committee_review` (đánh giá hội đồng đa góc nhìn).
4. **Pha Phản hồi (Deliver):** `synthesizer` (tổng hợp kết quả, loại bỏ trùng lặp), `safety_validator` (kiểm tra an toàn, thêm khuyến cáo), `react_controller` (quyết định lặp nếu thiếu bằng chứng), `react_retrieval` (truy xuất bổ sung), `memory_proposals` (đề xuất sở thích người dùng).

Sáu tác tử chuyên biệt

Hệ thống có sáu tác tử, mỗi tác tử đảm nhận một lĩnh vực chuyên môn riêng:

- **Property Search Agent:** Tìm kiếm bất động sản theo yêu cầu. Sử dụng hybrid search (kết hợp SQL filtering và pgvector semantic search) trên bảng listings và chunks. Tự động trích xuất bộ lọc từ truy vấn tiếng Việt (khu vực, giá, diện tích, loại hình, số phòng).
- **Investment Advisor Agent:** Phân tích tiềm năng đầu tư. Đánh giá ROI, dòng tiền cho thuê, tỷ suất vốn hóa dựa trên dữ liệu thực tế từ thị trường. Kết quả luôn kèm khuyến cáo tài chính (financial disclaimer).
- **Market Analysis Agent:** Phân tích thị trường bất động sản. Cung cấp số liệu thống kê giá theo quận, xu hướng thị trường, snapshot giá trung bình (không phải chuỗi thời gian).
- **News Agent:** Tra cứu tin tức thị trường mới nhất. Tìm kiếm trong kho bài viết (articles) được thu thập hàng tuần.
- **Project Agent:** Tra cứu thông tin dự án bất động sản. Tìm kiếm thông tin chủ đầu tư, quy mô, vị trí, tiện ích.
- **Legal Advisor Agent:** Tư vấn pháp lý. Truy xuất từ kho kiến thức pháp lý được cập nhật hàng tháng, bao gồm các vấn đề: sổ đỏ, thủ tục sang tên, thuế phí, quy hoạch. Kết quả luôn kèm khuyến cáo pháp lý (legal disclaimer).

1.2.3. Mẫu kiến trúc Bảng đen (Blackboard Pattern)

Mẫu kiến trúc Bảng đen (Blackboard Pattern) là mẫu kiến trúc trong đó nhiều thành phần chuyên biệt (knowledge sources) cùng đọc và ghi vào một không gian dữ liệu chia sẻ chung gọi là “bảng đen”. Mỗi thành phần hoạt động độc lập, được kích hoạt khi dữ liệu phù hợp xuất hiện trên bảng đen, và đóng góp kết quả trở lại bảng đen. Mẫu này đặc biệt phù hợp với các bài toán không có thuật toán giải xác định trước, đòi hỏi sự phối hợp của nhiều chuyên gia.

Trong Agent Service, Blackboard được triển khai qua `agent_blackboard` dictionary trong `AgentGraphState`. Mỗi `specialist agent` có thể:

- **Ghi (Write):** Sau khi hoàn thành phân tích, agent ghi một `BlackboardEntry` chứa: `id`, `author` (tên agent), `type` (loại nội dung: `specialist_result`, `market_insight`, `risk_assessment`),

content (kết quả phân tích), evidence_ids (danh sách bằng chứng tham chiếu), confidence (low/medium/high), created_at_step.

- **Đọc (Read):** Các agent khác có thể đọc entries thông qua hàm `entries_by_author()`. Ví dụ: Investment Advisor đọc kết quả từ Market Analysis và Legal Advisor để có đánh giá toàn diện hơn.
- **Validation:** Evidence IDs trong mỗi entry được validate chéo với evidence_by_id trong state để đảm bảo tính toàn vẹn dữ liệu.

Mẫu Bảng đen cho phép các agent phối hợp mà không cần biết về nhau tại thời điểm thiết kế (loose coupling), đồng thời cho phép thêm agent mới mà không ảnh hưởng đến các agent hiện có.

1.2.4. Mẫu kiến trúc Đánh giá Hội đồng (Committee Review)

Mẫu Đánh giá Hội đồng (Committee Review) là kỹ thuật trong đó một quyết định hoặc phân tích được xem xét từ nhiều góc nhìn độc lập trước khi đưa ra kết luận cuối cùng. Kỹ thuật này giúp giảm thiểu thiên kiến (bias) và tăng độ tin cậy của kết quả, đặc biệt trong các bài toán có tính rủi ro cao như tư vấn đầu tư.

Trong Agent Service, Committee Review được kích hoạt sau Investment Model, xem xét investment_case từ ba góc nhìn:

- **Bull (Lạc quan):** Tập trung vào các yếu tố tích cực – tiềm năng tăng giá, vị trí đặc địa, tiện ích xung quanh, chủ đầu tư uy tín, xu hướng thị trường thuận lợi. Mức độ rủi ro được đánh giá là thấp.
- **Bear (Thận trọng):** Tập trung vào các yếu tố rủi ro – thanh khoản thấp, chi phí vốn cao, biến động thị trường, rủi ro pháp lý, chi phí phát sinh. Mức độ rủi ro được đánh giá là cao.
- **Skeptic (Hoài nghi):** Đặt câu hỏi về các giả định, yêu cầu thêm bằng chứng cho các claims chưa được xác minh, chỉ ra các yếu tố chưa được đề cập. Mức độ rủi ro được đánh giá là trung bình.

Mỗi góc nhìn đưa ra: stance (lập trường), claims (các lập luận kèm evidence_ids), confidence (độ tin cậy), risk_level (mức độ rủi ro), và suggested_actions (hành động đề xuất). Kết quả hội đồng được đưa vào synthesizer để tổng hợp cùng với các agent outputs khác, đảm bảo người dùng nhận được bức tranh toàn diện, cân bằng giữa cơ hội và rủi ro.

1.2.5. Mẫu Suy luận – Hành động (ReAct)

ReAct (Reasoning + Acting) là mẫu thiết kế cho phép tác tử AI xen kẽ giữa suy luận (reasoning) và hành động (acting) để giải quyết các nhiệm vụ phức tạp. Thay vì sinh câu trả lời ngay lập tức, tác tử thực hiện các bước: suy nghĩ về tình huống hiện tại → quyết định hành động cần thực hiện → thực thi hành động → quan sát kết quả → suy nghĩ tiếp cho đến khi đạt được câu trả lời thỏa đáng.

Trong Agent Service, ReAct được triển khai qua hai nút trong đồ thị:

- **react_controller:** Sau khi `safety_validator` hoàn thành, bộ điều khiển kiểm tra `state.warnings`. Nếu phát hiện các cảnh báo nghiêm trọng (thiếu bằng chứng, agent không có evidence hợp lệ), controller quyết định hành động tiếp theo:
 - `retrieve_more`: Yêu cầu truy xuất thêm với bộ lọc mở rộng hoặc domain bổ sung.
 - `run_specialist`: Yêu cầu chạy lại `specialist agents` với evidence mới.
 - `ask_clarification`: Yêu cầu người dùng làm rõ câu hỏi.
 - `finalize`: Chấp nhận kết quả hiện tại và chuyển đến `memory_proposals`.
- **react_retrieval:** Khi controller yêu cầu `retrieve_more`, nút này thực hiện truy xuất bổ sung với filters từ `react_decision`, merge kết quả vào evidence hiện có, và chuyển về `specialist_agents` để xử lý lại.

Số vòng lặp tối đa được giới hạn bởi `AGENT_REACT_MAX_ITERATIONS` (mặc định 2) để đảm bảo thời gian phản hồi trong giới hạn cho phép.

1.2.6. Véc-tơ đặc trưng và Tìm kiếm ngữ nghĩa

Véc-tơ đặc trưng (Embedding)

Véc-tơ đặc trưng là kỹ thuật chuyển đổi văn bản thành một véc-tơ số thực d -chiều trong không gian Euclid, sao cho các văn bản có ngữ nghĩa tương đồng sẽ nằm gần nhau trong không gian này. Khoảng cách giữa hai véc-tơ thường được đo bằng độ tương đồng cosin hoặc khoảng cách Euclid (L2).

Đồ án sử dụng mô hình **BGE-M3** (BAAI/bge-m3) – một mô hình embedding đa ngôn ngữ mã nguồn mở, hỗ trợ hơn 100 ngôn ngữ bao gồm tiếng Việt. Các đặc điểm kỹ thuật: không gian embedding 1024 chiều, normalize embeddings về unit vector, max sequence length 8192 tokens, batch size 16, retry 3 lần với exponential backoff. Mô hình được sử dụng cho cả indexing (tạo embedding cho dữ liệu) và querying (tạo embedding cho câu truy vấn) để đảm bảo tính nhất quán của không gian vector.

Tìm kiếm ngữ nghĩa (Semantic Search)

Tìm kiếm ngữ nghĩa sử dụng vector embedding để tìm các đoạn văn bản có nội dung tương đồng với câu truy vấn, ngay cả khi không có từ khóa chung. Đồ án sử dụng **pgvector** – extension của PostgreSQL – để lưu trữ và tìm kiếm vector. Cụ thể: kiểu dữ liệu `VECTOR(1024)`, độ đo cosine distance (`<=>` operator), chỉ mục HNSW (Hierarchical Navigable Small World) với tham số $m = 16$ và $ef_construction = 64$.

1.2.7. Tìm kiếm lai và Xếp hạng lại

Tìm kiếm lai (Hybrid Search)

Tìm kiếm lai kết hợp hai phương pháp tìm kiếm bổ trợ trong cùng một truy vấn SQL:

1. **Structured Filtering (SQL WHERE):** Áp dụng các bộ lọc có cấu trúc trên các trường quan hệ – `listing_type` (sale/rent), `property_type`, `city`, `district`, `price` (min/max), `area` (min/max), `bedrooms`.
2. **Semantic Search (pgvector kNN):** Sử dụng vector embedding để tìm các chunks có nội dung tương đồng ngữ nghĩa với câu truy vấn.

Hai phương pháp được kết hợp: đầu tiên lọc theo điều kiện WHERE trên bảng `listings`, sau đó JOIN với bảng `chunks` và sắp xếp theo cosine distance trên cột embedding. Kết quả là danh sách k bất động sản vừa thỏa mãn bộ lọc cứng, vừa có nội dung mô tả phù hợp ngữ nghĩa.

Reranking (Xếp hạng lại)

Sau khi có tập kết quả ứng viên từ hybrid search, kỹ thuật reranking được áp dụng để sắp xếp lại theo độ liên quan thực tế. Đồ án sử dụng **Cohere Rerank Multilingual v3.0** – mô hình Cross-Encoder chuyên dụng xử lý đồng thời cặp (query, document) qua cùng một mạng transformer, cho phép nắm bắt các mối tương tác tinh tế. Kết quả được sắp xếp lại theo relevance score và lấy `top_n = 5`. Reranking được áp dụng tùy chọn (khi `COHERE_API_KEY` được cấu hình).

Redis Caching

Để giảm độ trễ, hệ thống tích hợp Redis caching ở hai cấp độ: embedding cache (lưu vector của câu truy vấn, key là hash của query text + model name + dimension) và search result cache (lưu kết quả hybrid search, key là hash của query + filters).

1.3. Các công nghệ sử dụng

Phần này trình bày chi tiết các công nghệ được lựa chọn trong đồ án. Với mỗi công nghệ, em sẽ giới thiệu tổng quan, phân tích lý do lựa chọn thay vì các giải pháp thay thế, và làm rõ vấn đề cụ thể mà công nghệ đó giải quyết trong hệ thống.

1.3.1. FastAPI

FastAPI là web framework Python hiện đại xây dựng trên nền tảng Starlette (ASGI) và Pydantic, tận dụng type hints để cung cấp validation tự động, sinh tài liệu OpenAPI và hỗ trợ async/await. Đây là một trong những Python framework có hiệu năng cao nhất hiện nay, sánh ngang với các framework viết bằng Node.js và Go.

Em chọn FastAPI thay vì Django REST Framework hay Flask bởi ba lý do. Trước hết, kiến trúc bất đồng bộ của FastAPI đặc biệt phù hợp với bản chất I/O-intensive của hệ thống: mỗi yêu cầu từ người dùng cần đồng thời truy vấn PostgreSQL, gọi Redis, gọi Agent Service nội bộ và gọi Gemini API – tất cả đều có thể thực thi song song qua async/await mà không chặn luồng xử lý. SQLAlchemy 2.0 với asyncpg driver tận dụng tối đa connection pool, cho phép hàng chục truy vấn cơ sở dữ liệu diễn ra đồng thời. Thứ hai, cơ chế validation tự động qua Pydantic v2 giúp em giảm đáng kể code boilerplate, đồng thời tự động sinh tài liệu API tương tác qua Swagger UI tại /docs – cực kỳ hữu ích trong quá trình phát triển và kiểm thử. Thứ ba, hệ thống dependency injection cho phép tái sử dụng logic một cách sạch sẽ: database session, xác thực JWT, kiểm tra internal key đều được định nghĩa một lần và dùng lại qua Depends().

Trong đồ án, FastAPI được sử dụng cho cả ba dịch vụ: Backend API (cổng 8000) xử lý yêu cầu từ giao diện người dùng, Agent Service (cổng 8100) chạy đồ thị trạng thái LangGraph, và Pipeline Worker (cổng 8200) thực thi các tác vụ pipeline. Việc dùng cùng một framework cho toàn bộ hệ thống giúp đồng bộ kiến trúc, giảm đáng kể chi phí học tập và bảo trì.

1.3.2. Next.js 16

Next.js là React framework hỗ trợ Server-Side Rendering (SSR), Static Site Generation (SSG) và App Router. Phiên bản 16 kết hợp React 19 và Tailwind CSS v4 mang lại hiệu năng cao cùng trải nghiệm phát triển hiện đại.

Em chọn Next.js thay vì Create React App hay Vite thuần bởi đây là một nền tảng bất động sản – nơi khả năng hiển thị trên Google, Bing là yếu tố sống còn. SSR đảm bảo mọi trang danh sách và chi tiết bất động sản đều được render phía server, nội dung có sẵn cho crawler ngay từ lần tải đầu tiên. Bên cạnh đó, React Server Components mặc định giúp

giảm mạnh JavaScript gửi về trình duyệt – điều đặc biệt quan trọng với người dùng Việt Nam, nơi tốc độ mạng di động còn nhiều hạn chế. Hệ sinh thái phong phú với Tailwind CSS v4, App Router với layouts lồng nhau, loading states và error boundaries giúp em tổ chức code rõ ràng và phát triển nhanh.

Hệ thống bao gồm đầy đủ các trang: duyệt danh sách nhà đất bán (/nha-dat-ban) và cho thuê (/nha-dat-cho-thue), chi tiết bất động sản, thống kê thị trường (/thi-truong) với biểu đồ tương tác, tin tức (/tin-tuc), dự án (/du-an), đăng nhập (/dang-nhap), đăng ký (/dang-ky), và bảng điều khiển quản trị (/admin). File lib/api.ts định nghĩa typed API client giúp giao tiếp an toàn với Backend API qua JWT token.

1.3.3. PostgreSQL + pgvector

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở với hơn 30 năm phát triển, nổi tiếng về độ ổn định và hỗ trợ ACID đầy đủ. Extension pgvector bổ sung kiểu dữ liệu VECTOR cùng các phép toán tương đồng, biến PostgreSQL thành vector database mà không cần thêm một hệ thống riêng.

Em chọn PostgreSQL + pgvector thay vì các vector database chuyên dụng như ChromaDB, Pinecone hay Qdrant vì một lý do then chốt: khả năng tích hợp dữ liệu quan hệ và dữ liệu véc-tơ trong cùng một hệ thống. Dữ liệu quan hệ (listings, users, chats, projects, articles) và dữ liệu véc-tơ (chunks với embedding 1024 chiều) cùng nằm trong PostgreSQL, cho phép em thực hiện JOIN giữa bảng listings và chunks, lọc theo WHERE trên các trường quan hệ đồng thời với tìm kiếm cosine distance – tất cả trong một truy vấn SQL duy nhất. Điều này không chỉ giảm độ phức tạp vận hành (chỉ cần quản lý một CSDL thay vì hai) mà còn tận dụng được chỉ mục HNSW với tham số $m = 16$, $ef_construction = 64$ cho tốc độ truy vấn nhanh trên hàng trăm nghìn véc-tơ 1024 chiều.

Chính nhờ PostgreSQL + pgvector mà hệ thống tìm kiếm lai của đồ án trở nên khả thi: người dùng có thể lọc bất động sản theo tiêu chí cứng (thành phố, quận, giá, diện tích, số phòng) qua WHERE truyền thống, đồng thời tìm kiếm theo ngữ nghĩa (“căn hộ thoáng mát gần công viên”) qua cosine distance trên cột embedding. Kết quả trả về là những bất động sản vừa khớp bộ lọc cứng, vừa có nội dung mô tả phù hợp ngữ nghĩa với truy vấn.

1.3.4. LangGraph

LangGraph là thư viện của LangChain cho phép xây dựng ứng dụng đa tác tử có trạng thái dưới dạng đồ thị có hướng. Các khái niệm cốt lõi gồm StateGraph (đồ thị trạng thái với state object được truyền qua các node), Nodes (hàm xử lý cho từng bước), Edges (normal edge và conditional edge), và Checkpointing (lưu trạng thái để phục hồi và gỡ lỗi).

Em chọn LangGraph thay vì CrewAI hay AutoGen vì đồ thị trạng thái tường minh của nó. Với LangGraph, toàn bộ luồng xử lý được định nghĩa rõ ràng dưới dạng đồ thị có hướng: mỗi node là một bước độc lập, mỗi conditional edge quyết định đường đi tiếp theo dựa trên trạng thái hiện tại. Điều này khiến luồng xử lý trở nên minh bạch, dễ kiểm tra và dễ gỡ lỗi – khác hẳn với cách tiếp cận hội thoại tự do của CrewAI hay AutoGen. Hơn nữa, state object AgentGraphState lưu trữ toàn bộ quá trình xử lý từ request gốc, intent, retrieval plan, kết quả từng agent cho đến final response, sources, trace steps và warnings – cho phép em theo dõi và đánh giá chất lượng một cách toàn diện.

LangGraph giải quyết bài toán cốt lõi của đề án: điều phối quy trình xử lý đa bước, đa tác tử một cách có kiểm soát. Trong Agent Service, đồ thị 14 nút điều phối toàn bộ hành trình từ câu hỏi đến phản hồi qua bốn pha: Hiểu (context builder, readiness checker, router, clarification), Truy xuất (query understanding, retrieval planner), Suy luận (6 specialist agents song song, investment model, committee review), và Phản hồi (synthesizer, safety validator, ReAct controller, memory proposals).

1.3.5. Google Gemini 2.0 Flash

Google Gemini 2.0 Flash là mô hình ngôn ngữ lớn đa năng của Google, hỗ trợ đa ngôn ngữ (bao gồm tiếng Việt), được tối ưu cho tốc độ cao và chi phí thấp. Mô hình có khả năng sinh văn bản, phân loại ý định và – quan trọng nhất với đề án – trả về kết quả JSON có cấu trúc theo schema định trước.

Em chọn Gemini 2.0 Flash thay vì GPT-4o hay Claude 3.5 sau khi thử nghiệm nội bộ với các truy vấn bất động sản tiếng Việt. Gemini cho phản hồi tự nhiên, chính xác về ngữ pháp và ngữ nghĩa, phù hợp với ngữ cảnh địa phương, trong khi chi phí thấp hơn đáng kể so với GPT-4o – yếu tố quan trọng với ngân sách đề án sinh viên. Em còn tích hợp cơ chế theo dõi chi phí qua module `agent_service/llm/cost.py` với ngân sách hàng tháng `AGENT_LLM_MONTHLY_BUDGET_USD` để kiểm soát chặt chẽ.

Trong hệ thống, Gemini đảm nhiệm ba vai trò. Vai trò thứ nhất là phân loại ý định người dùng: khi được kích hoạt, Gemini phân tích truy vấn tiếng Việt và trả về JSON chứa intent cùng danh sách tác tử mục tiêu, cho độ chính xác cao hơn hẳn phương pháp khớp từ khóa với các truy vấn phức tạp. Vai trò thứ hai là sinh phản hồi chuyên biệt: mỗi tác tử nhận system prompt mô tả vai trò cùng ngữ cảnh truy xuất, sau đó Gemini sinh phản hồi phù hợp với lĩnh vực của tác tử đó. Vai trò thứ ba là đánh giá chất lượng: Gemini hoạt động như một “giám khảo”, chấm điểm phản hồi của chatbot qua năm tiêu chí (tính căn cứ, tính hữu ích, chất lượng trích dẫn, an toàn, tính đầy đủ của vết xử lý). Khi không có API key, hệ thống tự động chuyển sang keyword-based routing và template-based response, đảm bảo luôn hoạt

động được.

1.3.6. BGE-M3 Embedding

BGE-M3 (BAAI/bge-m3) là mô hình embedding đa ngôn ngữ mã nguồn mở từ BAAI, hỗ trợ hơn 100 ngôn ngữ bao gồm tiếng Việt. Mô hình tạo véc-tơ 1024 chiều từ văn bản đầu vào và chạy hoàn toàn cục bộ qua `sentence-transformers`, không phụ thuộc vào bất kỳ API bên ngoài nào.

Đây là lựa chọn then chốt của em cho bài toán tìm kiếm ngữ nghĩa tiếng Việt. Qua thử nghiệm, BGE-M3 cho chất lượng embedding vượt trội so với Gemini Embedding trên văn bản tiếng Việt – điều quyết định độ chính xác của toàn bộ hệ thống tìm kiếm. Quan trọng hơn, là mô hình mã nguồn mở chạy cục bộ, BGE-M3 không bị giới hạn về số lượng yêu cầu mỗi phút hay chi phí theo lượt sử dụng. Điều này đặc biệt quan trọng khi em cần tạo embedding cho hàng trăm nghìn chunks trong quá trình lập chỉ mục – một khối lượng công việc mà nếu dùng API trả phí sẽ tốn kém không tưởng. BGEEmbedder trong đồ án hỗ trợ encode theo batch (`batch_size=16`) và chạy bất đồng bộ qua `asyncio.to_thread()` để không chặn event loop của FastAPI.

Cùng một mô hình BGE-M3 được dùng cho cả indexing (tạo embedding cho dữ liệu) và querying (tạo embedding cho câu truy vấn), đảm bảo tính nhất quán của không gian véc-tơ: truy vấn và tài liệu được ánh xạ vào cùng không gian 1024 chiều, nơi khoảng cách cosin phản ánh chính xác độ tương đồng ngữ nghĩa.

1.3.7. Cohere Rerank

Cohere Rerank Multilingual v3.0 là mô hình xếp hạng lại chuyên dụng sử dụng kiến trúc Cross-Encoder. Không giống Bi-Encoder (mã hóa truy vấn và tài liệu riêng biệt rồi so sánh khoảng cách), Cross-Encoder xử lý đồng thời cặp (truy vấn, tài liệu) qua cùng một mạng transformer, nắm bắt được những tương tác tinh tế mà Bi-Encoder bỏ lỡ.

Em tích hợp Cohere Rerank như một bước tinh chỉnh cuối cùng trong pipeline tìm kiếm. Sau khi hybrid search thu hẹp không gian từ hàng trăm nghìn xuống vài chục ứng viên, reranker chấm điểm độ liên quan cho từng cặp (truy vấn, tài liệu) và giữ lại top 5 kết quả tốt nhất. Cross-Encoder cho độ chính xác cao hơn đáng kể so với Bi-Encoder, đặc biệt với những truy vấn phức tạp như “căn hộ cao cấp có hồ bơi, gần trung tâm thương mại, phù hợp cho thuê dài hạn”. Reranking được thiết kế dưới dạng tùy chọn qua biến môi trường `RERANK_PROVIDER` – hệ thống vẫn hoạt động tốt nếu không có Cohere API key, nhưng sẽ chính xác hơn khi được kích hoạt.

1.3.8. Redis 7

Redis là cơ sở dữ liệu key-value hoạt động hoàn toàn trên RAM, nổi tiếng với tốc độ truy xuất dưới 1ms và cơ chế TTL tự động. Em sử dụng Redis ở hai cấp độ cache để giảm độ trễ cho hệ thống.

Cấp độ thứ nhất là embedding cache: vector của câu truy vấn được lưu với key là hash của (query text + model name + dimension), giúp tránh phải encode lại cùng một câu truy vấn qua BGE-M3 nhiều lần – một thao tác tốn kém về mặt tính toán. Cấp độ thứ hai là search result cache: kết quả hybrid search được lưu với key là hash của (query + filters), phục vụ tức thì các truy vấn lặp lại mà không cần chạm đến PostgreSQL. Cơ chế TTL đảm bảo cache tự động làm mới, luôn phản ánh thông tin mới nhất mà không cần dọn dẹp thủ công. Ngoài ra, Redis còn được dùng để lưu session state cho chat.

1.3.9. Apache Airflow

Apache Airflow là nền tảng quản lý workflow mã nguồn mở, cho phép định nghĩa, lập lịch và giám sát các đường ống dữ liệu dưới dạng DAG (Directed Acyclic Graph). Mỗi DAG gồm nhiều task được tổ chức theo quan hệ phụ thuộc, với cơ chế retry và cảnh báo tích hợp.

Em chọn Airflow thay vì cron job thủ công vì đường ống dữ liệu của đồ án có nhiều bước phụ thuộc phức tạp: crawl URL trước khi crawl chi tiết, ingest vào database trước khi tạo embedding, đánh dấu active sau khi cập nhật. Airflow cho phép em định nghĩa các phụ thuộc này một cách tường minh qua cú pháp », tự động xử lý song song các nhánh độc lập, và retry đến 3 lần với exponential backoff (5 → 10 → 20 phút) khi thất bại.

Hệ thống quản lý bốn DAG chính. `daily_listings_dag` chạy hàng ngày lúc 02:00, thu thập và xử lý tin bán và cho thuê mới. `weekly_projects_dag` và `weekly_news_dag` lần lượt chạy vào Chủ nhật 03:00 và 04:00, thu thập dự án và tin tức. `monthly_legal_kb_dag` chạy vào ngày 1 hàng tháng lúc 05:00, xử lý lại toàn bộ tài liệu pháp lý. Toàn bộ metrics được ghi vào bảng `pipeline_runs` để theo dõi hiệu suất.

1.3.10. Playwright

Playwright là thư viện tự động hóa trình duyệt do Microsoft phát triển, hỗ trợ điều khiển Chromium, Firefox và WebKit qua một API thống nhất. Nó cho phép mô phỏng hành vi người dùng thực – click, scroll, điền form – và trích xuất dữ liệu từ cả những trang web động phức tạp nhất.

Em chọn Playwright thay vì BeautifulSoup/Scrapy hay Selenium vì một thực tế: `batdongsan.com.vn` sử dụng rất nhiều JavaScript cho lazy loading, infinite scroll và AJAX.

BeautifulSoup chỉ xử lý được HTML tĩnh – hoàn toàn bất lực trước những trang như vậy. Playwright điều khiển trình duyệt Chromium thực nên thực thi được mọi JavaScript, chờ nội dung tải xong rồi mới trích xuất. Hơn nữa, cơ chế stealth giúp crawler tránh bị phát hiện là bot – yếu tố sống còn để thu thập dữ liệu ổn định và lâu dài. Với 4 workers chạy song song, tốc độ thu thập vượt trội so với Selenium.

Crawler được em tổ chức thành năm module: `core/` (csv_writer, parser dùng chung), `sale/` (tin bán), `rent/` (tin cho thuê), `projects/` (dự án), và `news/` (tin tức). Mỗi module thực hiện hai bước: crawl danh sách URL rồi crawl chi tiết từng tin. Dữ liệu ghi ra CSV UTF-8 BOM, được deduplicate qua `product_id` và merge tự động.

1.3.11. Docker & Docker Compose

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ phụ thuộc vào một container nhẹ, độc lập. Docker Compose là công cụ điều phối nhiều container qua file YAML.

Em chọn Docker & Docker Compose vì đây là cách duy nhất để đảm bảo hệ thống sáu dịch vụ chạy nhất quán trên mọi môi trường – từ máy phát triển của em đến máy chấm của hội đồng. Toàn bộ phụ thuộc (Python packages, Node modules, system libraries) được đóng gói cùng ứng dụng, loại bỏ hoàn toàn vấn đề “trên máy em chạy được mà”. Docker Compose quản lý sáu dịch vụ độc lập (PostgreSQL+pgvector, Redis, Backend API, Agent Service, Pipeline Worker, Frontend) với healthcheck và `depends_on` đảm bảo khởi động đúng thứ tự: PostgreSQL và Redis khởi động trước, rồi đến Pipeline Worker và Agent Service, sau đó Backend API, và cuối cùng là Frontend.

Chỉ với một lệnh `docker compose up -d`, toàn bộ hệ thống sẵn sàng phục vụ – điều đặc biệt quan trọng trong bối cảnh đồ án tốt nghiệp, nơi khả năng tái tạo kết quả một cách dễ dàng là yêu cầu then chốt.

1.3.12. Các thư viện và công cụ hỗ trợ

Bên cạnh các công nghệ chính, đồ án còn sử dụng những thư viện và công cụ sau:

- **SQLAlchemy 2.0 (async)** – ORM bất đồng bộ qua `asyncpg` driver, giúp em thao tác cơ sở dữ liệu an toàn kiểu, hiệu năng cao, tránh SQL injection và giảm đáng kể code boilerplate so với raw SQL.
- **Pydantic v2** – Thư viện validation dữ liệu qua type hints, đảm bảo mọi dữ liệu đầu vào/đầu ra đều đúng định dạng, tự động sinh JSON Schema và tài liệu OpenAPI, phát hiện sớm lỗi ngay từ biên giới hệ thống.

- **bcrypt + PyJWT** – bcrypt băm mật khẩu với salt chống rainbow table; PyJWT tạo và xác thực JWT với HS256, thời hạn 24h. Cặp đôi này đảm bảo xác thực người dùng an toàn, không trạng thái giữa client và server.
- **HTTPX** – HTTP client bất đồng bộ hỗ trợ HTTP/2 và connection pooling. Em dùng HTTPX để gọi Cohere Rerank API và các internal service API mà không chặn event loop của FastAPI.
- **Prometheus + Grafana + Alertmanager** – Bộ ba giám sát: Prometheus thu thập metrics, Grafana trực quan hóa qua dashboard, Alertmanager cảnh báo khi vượt ngưỡng. Giúp em theo dõi số lượng chat request, độ trễ, tỷ lệ lỗi và trạng thái pipeline theo thời gian thực.
- **sentence-transformers** – Thư viện chạy mô hình BGE-M3 cục bộ với batch encoding, cho phép tạo embedding cho hàng trăm nghìn chunks tiếng Việt mà không tốn một xu chi phí API.

CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Chương này trình bày toàn bộ quá trình phân tích và thiết kế hệ thống. Nội dung bao gồm: phân tích yêu cầu chức năng và phi chức năng, thiết kế kiến trúc tổng thể theo mô hình dịch vụ vi mô với sáu dịch vụ độc lập, thiết kế chi tiết Dịch vụ Tác tử – trái tim AI của hệ thống với đồ thị trạng thái LangGraph 14 nút và sáu tác tử chuyên biệt, thiết kế cơ sở dữ liệu quan hệ kết hợp véc-tơ, chiến lược tìm kiếm lai và hệ thống giám sát toàn diện.

2.1. Phân tích yêu cầu

2.1.1. Yêu cầu chức năng

Hệ thống phục vụ ba nhóm đối tượng chính: người dùng cuối (người mua, người thuê, nhà đầu tư), quản trị viên hệ thống, và bản thân hệ thống (các tác vụ tự động). Dưới đây là các nhóm chức năng được phân tích chi tiết.

Nhóm chức năng dành cho người dùng cuối

Đây là nhóm chức năng cốt lõi, quyết định trải nghiệm của người dùng khi tương tác với nền tảng:

1. Xác thực và quản lý tài khoản:

- Đăng ký tài khoản mới với email và mật khẩu. Mật khẩu được hash bằng bcrypt trước khi lưu trữ.
- Đăng nhập bằng email/mật khẩu, hệ thống trả về JWT token có thời hạn 24 giờ.
- Người dùng chưa đăng nhập (anonymous) vẫn có thể sử dụng chatbot với giới hạn 20 lượt/ngày. Người dùng đã xác thực được nâng lên 200 lượt/ngày.
- Hồ sơ người dùng bao gồm: họ tên, email, số điện thoại, ảnh đại diện, quyền (is_admin, is_superuser).

2. Duyệt và tìm kiếm bất động sản:

- **Danh sách bất động sản bán:** Hiển thị dưới dạng lưới thẻ (card grid) với phân trang. Mỗi thẻ hiển thị: tiêu đề, giá, diện tích, địa chỉ (quận, thành phố), số phòng ngủ, loại tin.
- **Danh sách bất động sản cho thuê:** Tương tự như trang bán nhưng dành riêng cho tin cho thuê.

- **Bộ lọc nâng cao:** Người dùng có thể lọc theo 10+ tiêu chí: loại tin (bán/cho thuê), loại hình bất động sản (căn hộ, nhà riêng, đất nền, biệt thự,...), thành phố, quận/huyện, khoảng giá, diện tích, số phòng ngủ, số phòng tắm, hướng nhà.
- **Tìm kiếm theo từ khóa:** Hỗ trợ tìm kiếm full-text trên các trường tiêu đề, mô tả, địa chỉ, quận, thành phố.
- **Sắp xếp kết quả:** Theo giá tăng dần/giảm dần, diện tích tăng dần/giảm dần, mới nhất.

3. Xem chi tiết bất động sản:

- Trang chi tiết hiển thị đầy đủ thông tin của một bất động sản: tiêu đề, mô tả chi tiết, giá (dạng số + text), đơn giá theo m^2 , diện tích, địa chỉ đầy đủ (phường/xã, quận/huyện, tỉnh/thành phố), tọa độ địa lý (latitude, longitude).
- Thông số kỹ thuật: số phòng ngủ, số phòng tắm, số tầng, hướng nhà, mặt tiền, đường vào.
- Thông tin pháp lý và nội thất: tình trạng pháp lý (sổ đỏ, sổ hồng,...), tình trạng nội thất (đầy đủ, cơ bản, bàn giao thô).
- **Bất động sản tương tự:** Hệ thống tự động gợi ý các bất động sản có đặc điểm tương đồng (cùng khu vực, cùng phân khúc giá, cùng loại hình).

4. Thống kê và phân tích thị trường:

- **Tổng quan thị trường:** Hiện thị các chỉ số chính: tổng số tin đăng đang hoạt động, giá trung bình, diện tích trung bình, số lượng tin bán và cho thuê.
- **Top khu vực:** Bảng xếp hạng các thành phố/quận có nhiều tin đăng nhất, có thể lọc theo loại tin.
- **Thống kê giá theo quận:** Với mỗi quận trong một thành phố, hiển thị số lượng tin, giá trung bình, giá thấp nhất, giá cao nhất và đơn giá trung bình/ m^2 .
- **Phân bố loại hình:** Thống kê số lượng tin theo từng loại hình bất động sản.

5. Tương tác với chatbot tư vấn đa tác tử:

- **Chat widget nổi:** Luôn hiển thị ở góc phải dưới màn hình, có thể mở rộng/thu gọn.
- **Hội thoại đa lượt:** Hệ thống lưu trữ toàn bộ lịch sử hội thoại theo phiên (session). Người dùng có thể xem lại các phiên chat trước đó.

- **Phản hồi có trích dẫn nguồn:** Mỗi câu trả lời của chatbot đều kèm danh sách nguồn tham khảo (tên bất động sản, đường dẫn, điểm liên quan), cho phép người dùng kiểm chứng thông tin.
- **Gợi ý câu hỏi tiếp theo:** Sau mỗi phản hồi, chatbot đề xuất 2–3 câu hỏi liên quan.
- **Đánh giá chất lượng:** Người dùng có thể đánh giá phản hồi (thích/không thích) và để lại nhận xét, giúp cải thiện chất lượng hệ thống.
- **Cá nhân hóa:** Hệ thống ghi nhớ sở thích của người dùng (khu vực ưa thích, ngân sách, loại hình) và sử dụng chúng để cá nhân hóa phản hồi trong các phiên sau.

Nhóm chức năng chatbot đa tác tử – Chi tiết

Đây là nhóm chức năng cốt lõi tạo nên sự khác biệt của hệ thống so với các nền tảng bất động sản truyền thống. Hệ thống triển khai kiến trúc hai lớp: đường ống nội bộ trong Backend xử lý nhanh các truy vấn đơn giản, và Dịch vụ Tác tử (Agent Service) chạy đồ thị trạng thái LangGraph riêng biệt cho suy luận đa bước nâng cao.

1. Phân loại ý định người dùng (Intent Routing):

- Hệ thống tự động phân tích câu hỏi tiếng Việt để xác định người dùng đang cần: tìm kiếm bất động sản, phân tích thị trường, tư vấn pháp lý, tư vấn đầu tư, tra cứu dự án, hay cập nhật tin tức.
- **Ba cơ chế routing:** (1) Keyword-based – sử dụng regex và từ khóa tiếng Việt không dấu, hoạt động không cần API key; (2) LLM-based – sử dụng Google Gemini 2.0 Flash để phân loại với độ chính xác cao hơn; (3) Hybrid – kết hợp keyword cho tốc độ và LLM cho độ chính xác.
- Router có thể chọn nhiều agent cùng lúc nếu câu hỏi liên quan đến nhiều lĩnh vực.

2. Sáu tác tử chuyên biệt hoạt động song song:

- **Property Search Agent:** Tìm kiếm bất động sản theo yêu cầu – tự động trích xuất bộ lọc từ truy vấn (giá, diện tích, khu vực, loại hình, số phòng), thực hiện hybrid search kết hợp SQL filtering và pgvector semantic search, xếp hạng lại qua Cohere Rerank.
- **Investment Advisor Agent:** Phân tích đầu tư – đánh giá ROI, dòng tiền cho thuê, tỷ suất vốn hóa dựa trên dữ liệu thực tế từ thị trường, kèm theo khuyến cáo tài chính bắt buộc.

- **Market Analysis Agent:** Phân tích thị trường – cung cấp số liệu thống kê giá theo quận, xu hướng thị trường, snapshot giá trung bình (không phải chuỗi thời gian).
- **News Agent:** Tra cứu tin tức – tìm kiếm trong kho bài viết tin tức thị trường mới nhất.
- **Project Agent:** Tra cứu dự án – tìm kiếm thông tin chủ đầu tư, quy mô, vị trí, tiện ích của các dự án bất động sản.
- **Legal Advisor Agent:** Tư vấn pháp lý – truy xuất từ kho kiến thức pháp lý (sổ đỏ, thủ tục sang tên, thuế phí, quy hoạch), kèm theo khuyến cáo pháp lý bắt buộc.

3. Suy luận đầu tư chuyên sâu (Investment Model):

- Khi người dùng hỏi về đầu tư, hệ thống xây dựng mô hình tài chính với các chỉ số: giá bất động sản, dòng tiền cho thuê kỳ vọng, tỷ lệ vốn vay, lãi suất, kỳ hạn vay, chi phí vận hành, tỷ lệ trống.
- Tính toán các chỉ số đầu tư: ROI, NPV (Net Present Value), dòng tiền hàng năm, điểm hòa vốn.

4. Đánh giá hội đồng (Committee Review):

- Kết quả phân tích đầu tư được xem xét từ ba góc nhìn độc lập: bullish (lạc quan), bearish (thận trọng), và skeptical (hoài nghi).
- Mỗi góc nhìn đưa ra lập luận, mức độ rủi ro và hành động đề xuất dựa trên cùng bộ bằng chứng.

5. Cơ chế ReAct (Reasoning + Acting):

- Sau khi tổng hợp và kiểm tra an toàn, hệ thống tự đánh giá chất lượng phản hồi.
- Nếu phát hiện thiếu bằng chứng hoặc độ tin cậy thấp, bộ điều khiển ReAct quyết định: truy xuất thêm dữ liệu, chạy lại tác tử, hoặc yêu cầu người dùng làm rõ câu hỏi.
- Quá trình lặp tối đa 2 vòng, đảm bảo phản hồi cuối cùng đạt chất lượng cao nhất có thể.

6. Bảng đen liên tác tử (Agent Blackboard):

- Các tác tử có thể đọc và ghi vào một không gian chia sẻ chung (blackboard), cho phép tác tử này tận dụng kết quả phân tích của tác tử khác.

- Ví dụ: Investment Advisor có thể đọc kết quả từ Market Analysis và Legal Advisor để đưa ra đánh giá toàn diện hơn.
- Mỗi mục trên bảng đen được gán nhãn tác giả, loại nội dung, danh sách bằng chứng và độ tin cậy.

7. Tổng hợp và phản hồi (Synthesis):

- Khi nhiều agent cùng được kích hoạt, hệ thống tổng hợp kết quả thành một phản hồi duy nhất, mạch lạc, loại bỏ trùng lặp.
- Mỗi phản hồi đều được kiểm tra an toàn (safety validation): phải có khuyến cáo pháp lý cho Legal Advisor, khuyến cáo tài chính cho Investment Advisor.
- Nếu không có đủ bằng chứng, hệ thống trung thực thông báo “Chưa có đủ thông tin” thay vì bịa đặt.

8. Ghi nhớ và cá nhân hóa (Memory System):

- Chatbot tự động trích xuất sở thích người dùng từ hội thoại (thành phố, quận, loại hình, ngân sách) và tạo MemoryProposal.
- Các sở thích có độ tin cậy cao được tự động áp dụng. Các sở thích khác cần người dùng xác nhận qua API.
- Sở thích đã xác nhận được sử dụng để cá nhân hóa phản hồi trong các phiên chat sau.

9. Đánh giá chất lượng tự động (LLM Judge):

- Hệ thống tích hợp “giám khảo” LLM sử dụng Gemini để chấm điểm mỗi phản hồi theo năm tiêu chí: tính căn cứ (groundedness), tính hữu ích (helpfulness), chất lượng trích dẫn (citation_quality), an toàn (safety), và tính đầy đủ của vết xử lý (trace_completeness).
- Kết quả đánh giá được lưu vào bảng EvalRun để phân tích và cải thiện hệ thống liên tục.

Nhóm chức năng quản trị và tự động hóa

1. Thu thập dữ liệu tự động:

- **Crawl tin đăng hàng ngày:** Tự động thu thập tin bán và cho thuê mới từ batdongsan.com.vn vào 02:00 giờ Hồ Chí Minh. Cơ chế watermark đảm bảo chỉ crawl dữ liệu mới, tránh trùng lặp.

- **Crawl dự án hàng tuần:** Thu thập thông tin dự án bất động sản vào Chủ nhật hàng tuần.
- **Crawl tin tức hàng tuần:** Thu thập bài viết phân tích thị trường, tin tức bất động sản.
- **Cập nhật kiến thức pháp lý hàng tháng:** Xử lý lại toàn bộ tài liệu pháp lý, bao gồm cả HTML và PDF.

2. Xử lý dữ liệu (ETL Pipeline):

- **Làm sạch:** Chuẩn hóa giá (tỷ, triệu, triệu/tháng), diện tích (m²), phân loại loại hình bất động sản, chuẩn hóa địa chỉ.
- **Làm giàu:** Geocoding – chuyển địa chỉ thành tọa độ (latitude, longitude). Intent tagging – tự động gán nhãn nhu cầu dựa trên nội dung mô tả.
- **Chunking:** Chia văn bản thành các đoạn ngữ nghĩa (400 token, 80 token chông lán) với 4 loại: overview, description, location, intent_tags.
- **Embedding:** Tạo vector embedding 1024 chiều cho từng chunk sử dụng mô hình BGE-M3 với batch size 16.
- **Nạp dữ liệu:** Upsert vào PostgreSQL qua các mô-đun chuyên biệt cho từng loại dữ liệu.

3. Quản trị và giám sát hệ thống:

- **Admin Dashboard:** Giao diện quản trị với các tab: Pipeline Readiness (trạng thái các nguồn dữ liệu), Agent Traces (lịch sử xử lý với latency và status), Chat Feedback (phản hồi người dùng), Memory Proposals (các đề xuất đang chờ xác nhận).
- **Observability toàn diện:** Hệ thống tracing ghi lại toàn bộ luồng xử lý: intent, agents được chọn, thời gian từng bước, kết quả retrieval, LLM calls, errors. Mỗi request chatbot được lưu thành một AgentTrace với đầy đủ các AgentTraceStep.
- **Theo dõi chi phí LLM:** Hệ thống theo dõi số token sử dụng và chi phí ước tính cho mỗi lần gọi Gemini, có cơ chế ngân sách hàng tháng (AGENT_LLM_MONTHLY_BUD để kiểm soát chi phí.
- **Monitoring:** Prometheus + Grafana + Alertmanager cho giám sát hiệu năng và cảnh báo.

2.1.2. Yêu cầu phi chức năng

Bảng 2.1 tổng hợp các yêu cầu phi chức năng của hệ thống.

Tiêu chí	Yêu cầu chi tiết
Hiệu năng	• API listings phản hồi < 500ms với bộ lọc cơ bản, < 1000ms với bộ lọc phức tạp • Chatbot pipeline hoàn thành < 10s cho truy vấn thông thường, < 45s cho truy vấn phức tạp đa tác tử • Trang web tải trong < 3s (First Contentful Paint) trên kết nối 4G
Khả năng mở rộng	• Kiến trúc microservices: 6 dịch vụ độc lập, mỗi service có thể scale riêng • Database connection pool: 20 connections + 10 overflow • Redis caching giảm tải database cho truy vấn lặp lại • Airflow DAGs hỗ trợ parallel task execution

Bảo mật	• Xác thực JWT với HS256, token hết hạn sau 24h • Mật khẩu hash bằng bcrypt với salt • Chống SQL injection qua SQLAlchemy ORM (parameterized queries) • Chống XSS qua Next.js auto-escaping • CORS whitelist qua biến môi trường CORS_ORIGINS • Internal API giữa các service bảo vệ bằng AGENT_INTERNAL_KEY
Độ tin cậy	• Fallback mechanism 3 cấp: Agent Service → internal pipeline → thông báo an toàn • Retry policy cho embedding: 3 lần với exponential backoff • Retry policy cho Airflow tasks: 3 lần, 5–30 phút • Health check endpoints cho tất cả services • Ready check với 30s cache TTL trước khi truy xuất dữ liệu

Khả năng bảo trì	• Mã nguồn phân chia module rõ ràng theo domain • Docstring tiếng Anh cho tất cả hàm/class • Type hints đầy đủ (Python) và TypeScript (frontend) • Hơn 60 file test bao phủ toàn bộ module • ESLint cho frontend, compileall cho Python
Tương thích ngôn ngữ	• Giao diện người dùng: 100% tiếng Việt • Chatbot: hiểu và phản hồi bằng tiếng Việt • Embedding: BGE-M3 tối ưu cho hơn 100 ngôn ngữ bao gồm tiếng Việt • URL slugs: tiếng Việt không dấu

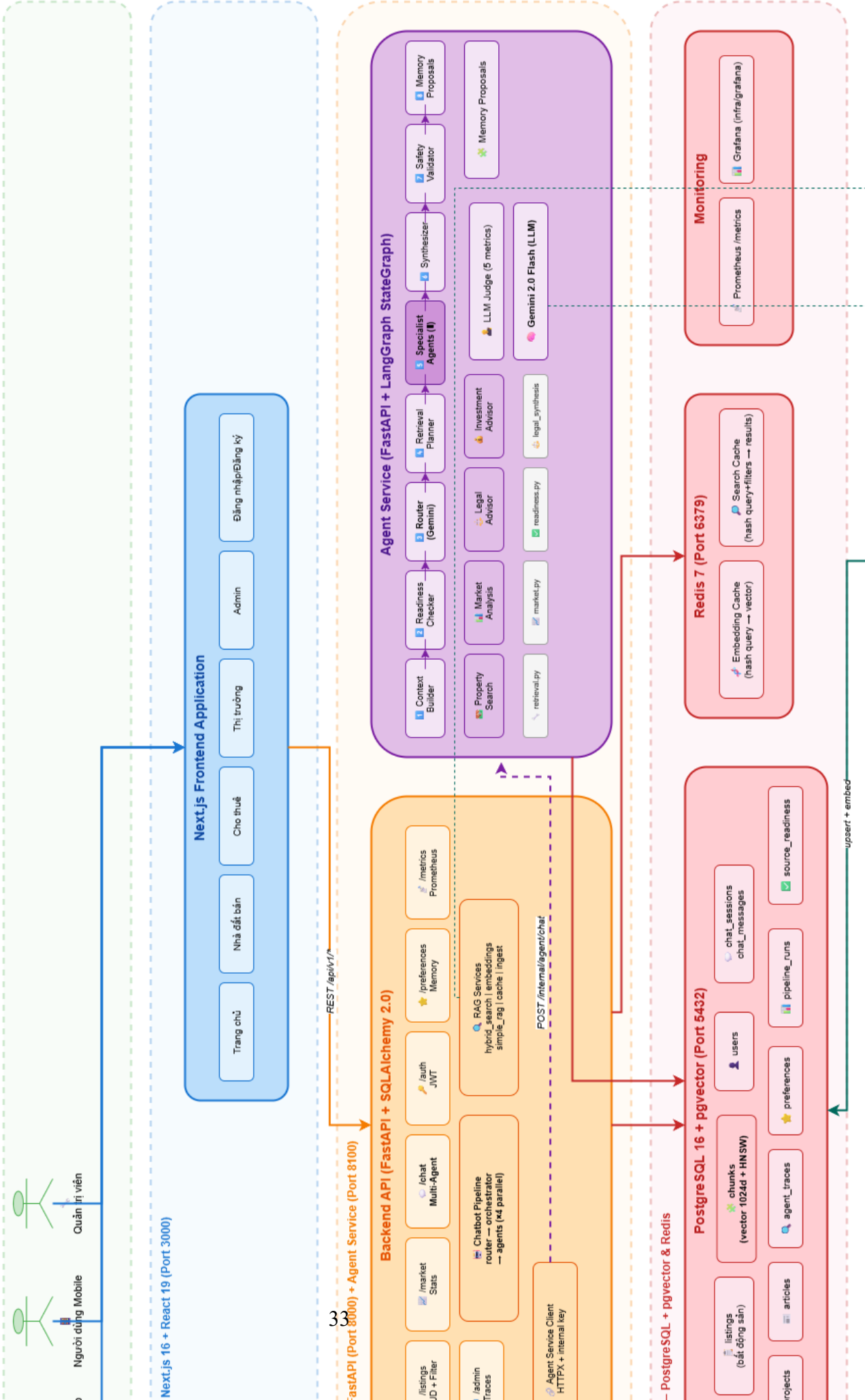
Bảng 2.1: Các yêu cầu phi chức năng của hệ thống

2.2. Thiết kế kiến trúc hệ thống

2.2.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc microservices với sáu dịch vụ độc lập, mỗi dịch vụ đảm nhận một trách nhiệm riêng biệt và giao tiếp với nhau qua REST API nội bộ. Hình 2.1 thể hiện kiến trúc tổng thể.

KIẾN TRÚC HỆ THỐNG REAL ESTATE CHATBOT PLATFORM



Tầng Người dùng (User Layer)

Người dùng tương tác với hệ thống qua trình duyệt web (desktop và mobile). Có hai nhóm người dùng chính:

- **Người dùng cuối:** Truy cập các trang duyệt bất động sản, xem thống kê thị trường và tương tác với chatbot tư vấn đa tác tử.
- **Quản trị viên:** Truy cập admin dashboard để giám sát pipeline, theo dõi agent traces và quản lý phản hồi người dùng.

Tầng Frontend – Next.js 16 (Cổng 3000)

Frontend được xây dựng với Next.js 16 và React 19, sử dụng App Router để tổ chức các trang. Các đặc điểm thiết kế chính:

- **Server-Side Rendering (SSR):** Các trang danh sách và chi tiết bất động sản được render phía server, đảm bảo SEO và thời gian tải nhanh.
- **React Server Components (RSC):** Mặc định sử dụng Server Components để giảm JavaScript gửi về client.
- **Tailwind CSS v4:** Utility-first CSS framework đảm bảo giao diện nhất quán, responsive trên mọi thiết bị.
- **Typed API Client:** File `lib/api.ts` cung cấp các hàm gọi backend với TypeScript type safety, tự động gắn JWT token vào header.
- **Chat Widget:** Thành phần nổi (floating) luôn hiển thị, cho phép người dùng tương tác với chatbot ở bất kỳ trang nào.

Frontend chỉ giao tiếp trực tiếp với Backend API qua REST. Frontend không được phép gọi trực tiếp Agent Service hay Pipeline Worker – đây là nguyên tắc thiết kế quan trọng để đảm bảo bảo mật và khả năng bảo trì.

Tầng Backend API – FastAPI (Cổng 8000)

Backend API là cổng giao tiếp chính của toàn bộ hệ thống, đảm nhận mọi request từ frontend. Backend được tổ chức thành 4 lớp:

1. **Lớp Models (ORM):** Định nghĩa 16+ bảng dữ liệu với SQLAlchemy 2.0 async. Mỗi model ánh xạ trực tiếp đến một bảng trong PostgreSQL. Các mối quan hệ (relationships)

được định nghĩa rõ ràng: Listing → Chunk (polymorphic), User → ChatSession → ChatMessage, User → UserPreference.

2. **Lớp Schemas (Pydantic v2):** Định nghĩa contract cho request/response. Pydantic v2 tự động validate dữ liệu đầu vào, sinh JSON Schema và tài liệu OpenAPI. Các schema được thiết kế tách biệt cho từng nghiệp vụ.
3. **Lớp Routers (API Endpoints):** Xử lý yêu cầu HTTP, gọi dịch vụ tương ứng và trả về phản hồi. Máy chủ có 9 mô-đun định tuyến: listings, market, projects, articles, chat, auth, preferences, metrics, admin. Các bộ định tuyến sử dụng dependency injection để nhận database session và xác thực người dùng.
4. **Lớp Services (Business Logic):** Chứa toàn bộ logic nghiệp vụ, được chia thành 3 nhóm chính:
 - **Chatbot Pipeline (services/chatbot/):** Pipeline multi-agent nội bộ với router (phân loại ý định), orchestrator (điều phối và tổng hợp), 4 agent chuyên biệt, context builder, memory manager. Đây là fallback khi Agent Service không khả dụng.
 - **RAG Services (services/rag/):** Các dịch vụ truy xuất thông tin: hybrid search (kết hợp SQL + vector + rerank + cache), embeddings (BGE-M3 wrapper với batch processing và retry), cache (Redis caching cho embedding và kết quả tìm kiếm).
 - **Agent Service Client (services/agent_service/):** HTTP client giao tiếp với Agent Service qua internal API, sử dụng internal key authentication. Bao gồm observability module để ghi nhận AgentTrace, AgentTraceStep, EvalRun vào database.

Tầng Agent Service – Trái tim AI (Công 8100)

Agent Service là dịch vụ vi mô quan trọng nhất của hệ thống, được thiết kế để xử lý các tác vụ trí tuệ nhân tạo phức tạp thông qua đồ thị trạng thái LangGraph. Dịch vụ này hoạt động độc lập với Backend API, giao tiếp qua REST với xác thực khóa nội bộ, đảm bảo isolation hoàn toàn về tài nguyên và lỗi.

Kiến trúc đồ thị trạng thái:

Đồ thị trạng thái LangGraph gồm 14 nút được tổ chức thành bốn pha xử lý chính, minh họa trong Hình 2.2.

Đồ thị Trạng thái LangGraph – 14 Nút, 4 Pha			
Pha 1: Hiểu	Pha 2: Truy xuất	Pha 3: Suy luận	Pha 4: Phản hồi
1. context_builder – Chuẩn hóa query – Ngữ cảnh hội thoại	4. query_understanding – Viết lại query – Trích xuất bộ lọc	6. specialist_agents – 6 agent song song 7. investment_model	9. synthesizer – Tổng hợp kết quả 10. safety_validator
2. readiness_checker – Kiểm tra nguồn DL – Cache 30s TTL	5. retrieval_planner – Lập kế hoạch – Hybrid search	– Phân tích tài chính 8. committee_review – Đánh giá hội đồng	– Khuyến cáo pháp lý – Khuyến cáo tài chính – Xác thực bằng chứng
3. router – Keyword/LLM/Hybrid – Chọn agent mục tiêu 3a. clarification – Làm rõ câu hỏi			11. react_controller – Quyết định lặp 12. react_retrieval – Truy xuất bổ sung 13. memory_proposals – Đề xuất sở thích

Hình 2.2: Tổng quan 14 nút trong đồ thị trạng thái LangGraph của Agent Service

Luồng xử lý chi tiết từng pha:

Pha 1 – Hiểu (Understand):

1. **context_builder** – Nhận AgentChatRequest từ Backend, chuẩn hóa truy vấn (strip accents, lowercase), compact lịch sử hội thoại (tối đa 5 lượt gần nhất), trích xuất sở thích người dùng từ UserPreference. Kết quả được đưa vào AgentGraphState để các nút sau sử dụng.
2. **readiness_checker** – Song song kiểm tra 4 nguồn dữ liệu (listings, projects, news, legal) qua SQL count. Mỗi nguồn được đánh giá “ready” nếu có cả parent records và chunk records. Kết quả được cache 30 giây để tránh query lặp lại. Nguồn không sẵn sàng sẽ bị loại khỏi retrieval plan.
3. **router** – Phân loại ý định và chọn danh sách tác tử. Hỗ trợ ba chế độ: `rule` (khớp từ khóa tiếng Việt không dấu với KEYWORDS_BY_AGENT), `llm` (gọi Gemini 2.0 Flash với prompt thiết kế riêng), `hybrid` (kết hợp cả hai). Router cũng phát hiện truy vấn thiếu thông tin và kích hoạt nhánh clarification.
4. **clarification** – Khi router phát hiện truy vấn không đủ thông tin (ví dụ: “tìm nhà” không có khu vực, ngân sách), nút này tạo câu hỏi làm rõ và kết thúc sớm đồ thị, trả về clarifying_question cho người dùng.

Pha 2 – Truy xuất (Retrieve):

5. **query_understanding** – Phân tích sâu ngữ nghĩa truy vấn. Ở chế độ `rule`: trích xuất bộ lọc có cấu trúc bằng regex (`listing_type`, `property_type`, `city`, `district`, `min/max price`, `min/max area`, `bedrooms`). Ở chế độ LLM (khi AGENT_QUERY_REWRITE gọi Gemini để viết lại truy vấn, mở rộng truy vấn (tối đa 3 biến thể), và suy luận bộ lọc ẩn. Bộ lọc được validate qua ALLOWED_FILTERS (18 trường được phép).
6. **retrieval_planner** – Lập kế hoạch truy xuất dựa trên ba yếu tố: (1) agents được chọn cần những domain nào (AGENT_RETRIEVAL_DOMAINS mapping), (2) nguồn dữ liệu nào đang sẵn sàng (từ readiness_checker), (3) bộ lọc đã trích xuất. Mỗi retrieval task được định nghĩa với: `task_id`, `domain`, `tool` (`search_listings/search_projects`), `query`, `filters`, `top_k` (mặc định 20), `rerank_top_k` (mặc định 5). Các task có dependency được giải quyết (ví dụ: `investment_advisor` phụ thuộc vào `market data`). Task được thực thi song song qua `asyncio.gather`, kết quả được chuẩn hóa thành Evidence objects với `source_identity`, `facts`, `matched_chunks`.

Pha 3 – Suy luận (Reason):

7. **specialist_agents** – Thực thi song song các tác tử được chọn từ router. Mỗi tác tử nhận evidence được gán riêng và sinh phản hồi chuyên biệt. Ở chế độ deterministic (mặc định): mỗi agent sử dụng template Python để định dạng kết quả từ evidence – mô tả từng bất động sản kèm giá, diện tích, vị trí và nguồn tham khảo. Ở chế độ LLM (khi AGENT_SPECIALIST_LLM_ENABLED): mỗi agent gọi Gemini với system prompt riêng và evidence context, cho phản hồi tự nhiên và có chiều sâu hơn. Kết quả mỗi agent được chuẩn hóa thành SpecialistResult với status, content, evidence_ids_used, confidence, warnings, sources. Các agent đồng thời ghi vào Agent Blackboard – không gian chia sẻ cho phép các agent đọc kết quả của nhau.
8. **investment_model** – Chỉ chạy khi investment_advisor được kích hoạt. Xây dựng mô hình đầu tư từ evidence đa miền: property evidence (giá, diện tích, vị trí), market evidence (giá thuê trung bình, xu hướng), legal evidence (tình trạng pháp lý), project evidence (thông tin chủ đầu tư), news evidence (tin tức liên quan). Tính toán các chỉ số tài chính với giả định mặc định (equity_ratio=0.4, loan_ratio=0.6, interest_rate_annual=0.1, loan_term_years=20, vacancy_months=1, operating_cost_ratio=0) giá trị đầu tư, dòng tiền hàng năm, NPV, ROI. Kết quả được cấu trúc thành investment_case (property_summary, market_summary, legal_summary, project_summary, news_summary, missing_evidence) và investment_metrics (từng chỉ số kèm value và unit).
9. **committee_review** – Đánh giá investment_case từ ba góc nhìn độc lập dựa trên cùng bộ evidence từ agent_blackboard:
- **Bull (Lạc quan):** Nhấn mạnh tiềm năng tăng giá, vị trí đắc địa, tiện ích xung quanh, chủ đầu tư uy tín. Mức độ rủi ro: thấp.
 - **Bear (Thận trọng):** Nhấn mạnh rủi ro thanh khoản, chi phí vốn cao, biến động thị trường, rủi ro pháp lý. Mức độ rủi ro: cao.
 - **Skeptic (Hoài nghi):** Đặt câu hỏi về các giả định, yêu cầu thêm bằng chứng, chỉ ra các yếu tố chưa được xác minh. Mức độ rủi ro: trung bình.

Mỗi góc nhìn đưa ra stance, claims (kèm evidence_ids), confidence, risk_level, và suggested_actions. Kết quả hội đồng được đưa vào synthesizer để tổng hợp cùng với các agent outputs khác.

Pha 4 – Phản hồi (Deliver):

10. **synthesizer** – Tổng hợp tất cả kết quả từ các agent, investment_model và committee_review thành một phản hồi cuối cùng. Quy trình: (1) thu thập tất cả agent outputs; (2) deduplicate warnings theo stable key (code+domain+message); (3) deduplicate sources theo type+id; (4) validate evidence references – mọi evidence được trích dẫn phải tồn tại trong evidence_for_agent; (5) nếu không có evidence nào, fallback “Chưa có đủ thông tin để trả lời câu hỏi này”; (6) nếu AGENT_SPECIALIST_LLM_ENABLED gọi Gemini synthesis prompt để tạo phản hồi mạch lạc; nếu không, ghép nối deterministic outputs. Kết quả bao gồm final_response, suggested_actions, sources, claims (mỗi claim kèm evidence_ids).
11. **safety_validator** – Kiểm tra an toàn phản hồi trước khi gửi đến người dùng. Các quy tắc: (1) nếu legal_advisor được chạy nhưng phản hồi không chứa legal disclaimer, thêm cảnh báo “legal_disclaimer_missing”; (2) nếu investment_advisor được chạy nhưng phản hồi không chứa financial disclaimer, thêm cảnh báo “financial_disclaimer_missing”; (3) các agent có nguồn dữ liệu (SOURCE_BACKED_AGENTS: legal_advisor, news_agent, project_agent, property_search) phải có ít nhất một evidence tham chiếu, nếu không thêm cảnh báo “agent_answer_missing_valid_evidence”. Cảnh báo được thêm vào state.warnings và hiển thị trong trace.
12. **react_controller** – Điều khiển vòng lặp phản ứng. Chỉ hoạt động khi AGENT_REACT_ENABLED. Kiểm tra state.warnings: nếu có cảnh báo nghiêm trọng (final_response_missing_sources, agent_answer_missing_valid_evidence), quyết định hành động tiếp theo: retrieve_more (truy xuất thêm với bộ lọc mở rộng), run_specialist (chạy lại specialist agents), ask_clarification (yêu cầu người dùng làm rõ), hoặc finalize (chấp nhận kết quả hiện tại). Số vòng lặp tối đa: AGENT_REACT_MAX_ITERATIONS (mặc định 2).
13. **react_retrieval** – Thực hiện truy xuất bổ sung khi react_controller yêu cầu. Sử dụng filters từ react_decision để mở rộng hoặc điều chỉnh retrieval plan. Kết quả được merge vào evidence_by_id và evidence_for_agent hiện có, sau đó quay lại specialist_agents.
14. **memory_proposals** – Trích xuất sở thích người dùng từ query và filters đã xác định. Mapping FILTER_TO_MEMORY_KEY: city → preferred_city, district → preferred_district, property_type → preferred_property_type, listing_type → listing_type, bedrooms → bedrooms, max_price → max_budget, min_price → min_budget. Mỗi proposal có confidence (0.72–0.82) và requires_user_confirmation=True.

Proposal được trả về trong AgentChatResponse để Backend lưu vào bảng memory_proposals và hiển thị cho người dùng xác nhận.

Cấu trúc dữ liệu trạng thái (AgentGraphState):

Toàn bộ trạng thái của đồ thị được lưu trong AgentGraphState – một TypedDict với hơn 40 trường, bao gồm:

- **Đầu vào:** request (AgentChatRequest với message, session_id, user_id, conversation_context, preferences).
- **Trung gian:** normalized_query, intent, agents_to_run, routing_filters, query_understanding, readiness, retrieval_plan, evidence_by_id, evidence_for_agent, agent_results, agent_blackboard, investment_case, investment_assumptions, investment_metrics, committee_review.
- **Điều khiển:** needs_clarification, force_deterministic, react_iteration, react_decision, react_actions.
- **Đầu ra:** final_response, sources (list[AgentSource]), suggested_actions, memory_proposals, trace_steps, warnings.

Cơ chế đặc biệt:

Agent Blackboard: Không gian chia sẻ giữa các tác tử, triển khai qua blackboard dictionary trong AgentGraphState. Mỗi entry có: id, author (tên agent), type (specialist_result, market_insight, risk_assessment,...), content, evidence_ids, confidence (low/medium/high), created_at_step. Các agent có thể đọc entry của nhau qua entries_by_author(). Investment Advisor đặc biệt tận dụng blackboard để tổng hợp insight từ Market Analysis và Legal Advisor.

Evidence Normalization: Mọi kết quả retrieval được chuẩn hóa thành Evidence objects với: evidence_id (duy nhất), retrieval_task_id (nguồn gốc), domain (property/project/news/legal/market), source_type, source_identity (stable key type:id), record (dữ liệu gốc), facts (title, price, location,...), source (AgentSource với type, id, title, url, score), matched_chunks (kèm vector_distance, rerank_score, final_score), retrieved_for (danh sách agent), assigned_to (danh sách agent được phân bổ), warnings.

Cost Tracking: Mỗi lần gọi Gemini được ghi nhận: input_tokens, output_tokens, estimated_cost_usd. Hệ thống kiểm tra AGENT_LLM_MONTHLY_BUDGET_USD trước mỗi lần gọi. Nếu vượt ngân sách, LLM bị vô hiệu hóa và hệ thống tự động chuyển sang deterministic mode.

LLM Judge: Endpoint /internal/agent/evaluate cho phép đánh giá chất lượng phản hồi qua 5 metrics: groundedness (phản hồi có được hỗ trợ bởi nguồn không),

helpfulness (có trực tiếp giúp người dùng không), citation_quality (nguồn có liên quan và rõ ràng không), safety (có khẳng định pháp lý/tài chính tuyệt đối không), trace_completeness (trace có đầy đủ agents, steps, warnings không). Judge prompt được thiết kế riêng cho tiếng Việt, yêu cầu JSON output với scores và rationales.

Tầng Pipeline Worker – Xử lý dữ liệu tự động (Cổng 8200)

Pipeline Worker là dịch vụ vi mô chuyên trách các tác vụ xử lý dữ liệu nặng, tách biệt khỏi Backend và Agent Service để tránh ảnh hưởng đến hiệu năng phục vụ người dùng.

Kiến trúc: Pipeline Worker chạy FastAPI trên cổng 8200, cung cấp internal API endpoints:

- POST /internal/pipeline/crawler – Thực thi module crawler (sale, rent, projects, news) qua subprocess. Nhận module name, args, timeout (mặc định 7200s).
- POST /internal/pipeline/ingest/listings – Nạp dữ liệu từ CSV vào PostgreSQL qua data_pipeline ingestors.
- POST /internal/pipeline/maintenance/cleanup-chunks – Dọn dẹp chunks cũ, giải phóng dung lượng.
- GET /internal/pipeline/health – Kiểm tra trạng thái dịch vụ.

Tất cả endpoints yêu cầu xác thực X-Internal-Agent-Key. Worker chạy các tác vụ dưới dạng subprocess Python, capture stdout/stderr, và trả về kết quả đồng bộ. Các module được gọi qua `python -m <module>` với PYTHONPATH được thiết lập đúng.

Tầng Dữ liệu (Data Layer)

PostgreSQL 16 + pgvector (Cổng 5432):

PostgreSQL được chọn làm cơ sở dữ liệu chính nhờ khả năng lưu trữ đồng thời dữ liệu quan hệ và dữ liệu véc-tơ thông qua extension pgvector. Điều này giúp thực hiện hybrid search trong cùng một truy vấn SQL: JOIN giữa bảng listings và bảng chunks, WHERE cho bộ lọc có cấu trúc, ORDER BY cosine distance cho tìm kiếm ngữ nghĩa.

Dữ liệu quan hệ gồm 16+ bảng: listings (30+ trường), projects, articles, users, chat_sessions, chat_messages, chunks, agent_traces, agent_trace_steps, agent_llm_calls, agent_retrieval_events, eval_runs, pipeline_runs, source_readiness, user_preferences, memory_proposals, chat_feedback.

Dữ liệu véc-tơ được lưu trong bảng chunks với cột embedding VECTOR(1024). Chỉ mục HNSW với tham số $m = 16$ và $ef_construction = 64$ đảm bảo tốc độ tìm kiếm k-NN

nhANH. Thiết kế đa hình (polymorphic) qua cặp cột (parent_type, parent_id) cho phép một bảng chunks phục vụ nhiều loại dữ liệu nguồn.

Redis 7 (Cổng 6379):

Redis được sử dụng làm bộ nhớ đệm ở hai cấp độ: embedding cache (lưu vector của câu truy vấn, tránh encode lại) và search result cache (lưu kết quả hybrid search cho truy vấn lặp lại).

Tầng Thu thập & Xử lý dữ liệu

Crawlers (Playwright + Stealth):

Hệ thống crawler được tổ chức thành 5 module:

- `crawler/core/`: Các thành phần dùng chung – CSV writer (với dedup), base parser.
- `crawler/sale/`: Crawl tin bán (2 bước: crawl URLs → crawl details).
- `crawler/rent/`: Crawl tin cho thuê.
- `crawler/projects/`: Crawl thông tin dự án bất động sản.
- `crawler/news/`: Crawl tin tức và bài viết phân tích thị trường.

Crawler sử dụng Playwright với chế độ stealth để tránh bị phát hiện, hỗ trợ 4 workers song song. Dữ liệu được ghi ra file CSV UTF-8 BOM và deduplicate dựa trên product_id.

Data Pipeline (ETL + Embedding):

Pipeline xử lý dữ liệu gồm 5 bước:

1. **Clean (`clean.py`):** Chuẩn hóa giá, diện tích, phân loại loại hình, chuẩn hóa địa chỉ.
2. **Enrich (`enrich.py`):** Geocoding (địa chỉ → tọa độ), Intent Tagging (gán nhãn nhu cầu).
3. **Chunk (`chunk.py`):** Chia văn bản thành các đoạn ngữ nghĩa 400 token, overlap 80 token, 4 loại: overview, description, location, intent_tags.
4. **Embed (`embed.py`):** Tạo vector embedding 1024 chiều bằng BGE-M3, batch size 16, retry 3 lần.
5. **Load (`load_db.py` + `ingestors/`):** Upsert vào PostgreSQL qua các ingestor module chuyên biệt.

Apache Airflow Scheduler:

Toàn bộ pipeline được lập lịch và giám sát bởi Apache Airflow với 4 DAGs:

- **daily_listings_dag (02:00 HCM):** Crawl + xử lý tin đăng mới hàng ngày.
- **weekly_projects_dag (Chủ nhật 03:00):** Crawl + xử lý dự án.
- **weekly_news_dag (Chủ nhật 04:00):** Crawl + xử lý tin tức.
- **monthly_legal_kb_dag (Ngày 1, 05:00):** Xử lý lại kho kiến thức pháp lý.

2.3. Thiết kế cơ sở dữ liệu

2.3.1. Mô hình quan hệ

Hệ thống sử dụng PostgreSQL 16 với extension pgvector. Mô hình dữ liệu được thiết kế xoay quanh 4 nhóm bảng chính:

Nhóm bảng nghiệp vụ (Business Tables)

- **listings – Bảng bất động sản:** Bảng quan trọng nhất với hơn 30 trường, lưu toàn bộ thông tin của một tin đăng. Khóa chính id (INTEGER). Mỗi listing có product_id duy nhất từ nguồn crawl để deduplicate. Các chỉ mục trên: listing_type, city, district, property_type, price, area, bedrooms, post_date, is_active.
- **projects – Bảng dự án:** Lưu thông tin dự án: tên, chủ đầu tư, vị trí, quy mô, khoảng giá, trạng thái, loại hình, tiện ích, mô tả.
- **articles – Bảng bài viết:** Lưu tin tức và kiến thức pháp lý: tiêu đề, nội dung, category (news/legal), nguồn, ngày đăng, URL gốc.
- **users – Bảng người dùng:** Lưu thông tin tài khoản: email (unique), mật khẩu đã hash, họ tên, số điện thoại, avatar, quyền.

Nhóm bảng hội thoại (Chat Tables)

- **chat_sessions – Phiên hội thoại:** id UUID, liên kết đến user (có thể NULL cho anonymous), tiêu đề tự động sinh.
- **chat_messages – Tin nhắn:** Mỗi tin nhắn thuộc về một session, role (user/assistant/system), nội dung, agent đã xử lý, metadata JSONB (sources, citations, search context).

Nhóm bảng vector và tìm kiếm (Vector & Retrieval)

- **chunks – Đoạn văn bản đã embedding:** Thiết kế polymorphic với `parent_type` (listing/project/area) và `parent_id`. Mỗi chunk có `chunk_type` (overview/description/location/intent_tags/legal), `text`, `embedding VECTOR(1024)`. HNSW index trên cột `embedding` với cosine distance operator.

Nhóm bảng giám sát & vận hành (Observability Tables)

- **agent_traces:** Lưu vết xử lý của mỗi request chatbot: `request_id`, `session_id`, `intent`, `agents_used`, `trace_summary`, `full_trace` (JSONB), `latency_ms`, `status` (success/partial/error), `graph_version`, `prompt_version`, `model_name`.
- **agent_trace_steps:** Chi tiết từng bước trong trace: `step_name`, `status`, `latency_ms`, `input/output` JSONB, `error`.
- **agent_llm_calls:** Log mỗi lần gọi LLM: `node_name`, `model_name`, `latency_ms`, `input_tokens`, `output_tokens`, `estimated_cost_usd`.
- **agent_retrieval_events:** Log mỗi sự kiện truy xuất: `tool_name`, `domain`, `filters`, `result_count`, `latency_ms`.
- **eval_runs:** Lưu kết quả đánh giá tự động: `scores` (5 metrics), `rationales`, `judge_model`, `graph_version`.
- **pipeline_runs:** Log mỗi lần chạy Airflow DAG.
- **source_readiness:** Trạng thái sẵn sàng của từng nguồn dữ liệu.
- **user_preferences:** Sở thích người dùng dạng key-value JSONB.
- **memory_proposals:** Đề xuất sở thích từ agent (trạng thái: pending/accepted/rejected).
- **chat_feedback:** Phản hồi của người dùng về chất lượng chat (rating, comment).

2.3.2. Chiến lược indexing

- **B-tree indexes cho listings:** `listing_type`, `city`, `district`, `property_type`, `price`, `area`, `bedrooms`, `post_date`, `is_active` – tối ưu các truy vấn lọc và sắp xếp phổ biến.
- **HNSW index cho chunks:** Trên cột `embedding` với `vector_cosine_ops`, $m = 16$, $ef_construction = 64$ – cho phép tìm kiếm k-NN trong không gian 1024 chiều với độ phức tạp $O(\log N)$.

- **Composite index cho polymorphic queries:** (parent_type, parent_id) trên bảng chunks – truy vấn nhanh tất cả chunks thuộc về một thực thể.
- **Index cho observability:** agent_traces (session_id, created_at), agent_trace_steps (trace_id) – hỗ trợ truy vấn admin dashboard.

2.4. Thiết kế giao diện người dùng

Hệ thống có giao diện người dùng web responsive, tối ưu cho cả desktop và mobile, được xây dựng với Next.js 16 và Tailwind CSS v4. Các trang chính bao gồm:

- **Trang chủ (/):** Giới thiệu nền tảng, thanh tìm kiếm nổi bật, các danh mục phổ biến, và chỉ số thị trường tổng quan.
- **Nhà đất bán (/nha-dat-ban):** Danh sách bất động sản bán dạng lưới thẻ với bộ lọc nâng cao và phân trang.
- **Nhà đất cho thuê (/nha-dat-cho-thue):** Tương tự nhưng cho tin cho thuê.
- **Thị trường (/thi-truong):** Biểu đồ tương tác hiển thị thống kê giá, phân bố loại hình, top khu vực.
- **Dự án (/du-an):** Danh sách dự án bất động sản với bộ lọc.
- **Tin tức (/tin-tuc):** Danh sách bài viết tin tức và phân tích thị trường.
- **Đăng nhập (/dang-nhap) / Đăng ký (/dang-ky):** Form xác thực người dùng.
- **Admin Dashboard (/admin):** Giao diện quản trị với các tab: Pipeline Readiness, Agent Traces, Chat Feedback, Memory Proposals.

Thành phần **Chat Widget** là một floating button luôn hiển thị ở góc phải dưới mọi trang. Khi click, widget mở rộng thành cửa sổ chat với: ô nhập liệu, lịch sử hội thoại (scrollable), hiển thị sources dưới dạng accordion, nút đánh giá (thích/không thích), và suggested actions. Widget gọi API `/api/v1/chat` qua typed client `lib/api.ts` với JWT token tự động đính kèm.

CHƯƠNG 3. TRIỂN KHAI VÀ KIỂM THỬ

Chương này trình bày chi tiết quá trình triển khai hệ thống lên môi trường vận hành và chiến lược kiểm thử toàn diện được áp dụng trong đề án. Phần đầu mô tả kiến trúc triển khai với Docker Compose cho sáu dịch vụ, cấu hình chi tiết từng dịch vụ, quy trình xây dựng và chạy hệ thống. Phần tiếp theo trình bày chiến lược kiểm thử đa tầng: từ kiểm thử đơn vị, kiểm thử tích hợp đến kiểm thử hệ thống. Cuối cùng, chương tổng hợp kết quả kiểm thử, các vấn đề đã phát hiện và hướng khắc phục.

3.1. Triển khai hệ thống với Docker

3.1.1. Kiến trúc triển khai

Hệ thống được triển khai theo mô hình container hóa với Docker Compose, bao gồm 6 service chính hoạt động trong cùng một mạng nội bộ (internal network). Mỗi service được đóng gói trong một Docker image riêng, đảm bảo tính nhất quán giữa môi trường phát triển và môi trường vận hành. Hình 3.1 mô tả tổng quan kiến trúc triển khai.

Docker Compose – Nền tảng Tư vấn Bất động sản					
Frontend	Backend	Agent Svc	Pipeline	PostgreSQL	Redis
Next.js 16	FastAPI	FastAPI+LangGraph	FastAPI	16 + pgvector	7 Alpine
Cổng 3000	Cổng 8000	Cổng 8100	Cổng 8200	Cổng 5432	Cổng 6379
Mạng nội bộ: realestate_network					
Ổ đĩa: pgdata, redisdata					

Hình 3.1: Kiến trúc triển khai Docker Compose với 6 dịch vụ

3.1.2. Cấu hình Docker Compose

Toàn bộ hệ thống được định nghĩa trong file `docker-compose.yml` tại thư mục gốc. Cấu hình này đảm bảo:

- **Khởi động có thứ tự:** Các dịch vụ khởi động theo đúng thứ tự phụ thuộc. PostgreSQL và Redis khởi động trước, sau đó đến Pipeline Worker và Agent Service, rồi Backend API, và cuối cùng là Frontend. Cơ chế `depends_on` kết hợp với `condition: service_healthy` đảm bảo dịch vụ chỉ khởi động khi các phụ thuộc đã sẵn sàng.
- **Kiểm tra trạng thái (Healthcheck):** Mỗi dịch vụ được cấu hình healthcheck riêng:
 - **PostgreSQL:** `pg_isready -U admin -d realestate` – kiểm tra mỗi 10 giây, tối đa 5 lần.

- **Redis:** `redis-cli ping` – kiểm tra mỗi 10 giây, tối đa 5 lần.
- **Agent Service:** Gọi `/internal/agent/health` với internal key – kiểm tra mỗi 10 giây, tối đa 12 lần, 30 giây khởi động ban đầu (cho phép tải mô hình BGE-M3 ~2GB).
- **Pipeline Worker:** Gọi `/internal/pipeline/health` – kiểm tra mỗi 10 giây, tối đa 12 lần, 30 giây khởi động.
- **Quản lý dữ liệu bền vững:** Hai ổ đĩa Docker: `pgdata` (toàn bộ dữ liệu PostgreSQL + chỉ mục HNSW) và `redisdata` (dữ liệu cache Redis). Các ổ đĩa tồn tại độc lập với vòng đời container.
- **Biến môi trường:** Tất cả cấu hình nhạy cảm được truyền qua biến môi trường từ tệp `.env`, không được gán cứng trong cấu hình Docker.

3.1.3. Cấu hình chi tiết từng dịch vụ

PostgreSQL + pgvector

```
postgres:
  image: pgvector/pgvector:pg16
  container_name: realestate_postgres
  environment:
    POSTGRES_DB: realestate
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data
  ports:
    - "5432:5432"
```

PostgreSQL được triển khai với image `pgvector/pgvector:pg16`, tích hợp sẵn extension `pgvector`. Extension được tự động kích hoạt qua SQLAlchemy trong quá trình khởi tạo schema (hàm `init_db()`).

Redis

```
redis:
  image: redis:7-alpine
  container_name: realestate_redis
```

```
ports:
  - "6379:6379"
volumes:
  - redisdata:/data
```

Redis sử dụng image Alpine nhẹ (~30MB). Đảm nhận caching embedding vector và kết quả hybrid search.

Agent Service

```
agent-service:
  build:
    context: .
    dockerfile: agent_service/Dockerfile
  container_name: realestate_agent_service
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  env_file: .env
  environment:
    DATABASE_URL: postgresql+asyncpg://admin:...@postgres:5432/realestate
    REDIS_URL: redis://redis:6379/0
    AGENT_INTERNAL_KEY: ${AGENT_INTERNAL_KEY}
    GEMINI_API_KEY: ${GEMINI_API_KEY}
    GEMINI_MODEL: gemini-2.0-flash
    GEMINI_JUDGE_MODEL: gemini-2.0-flash
    HF_EMBEDDING_MODEL: BAAI/bge-m3
```

Dịch vụ Tác tử được xây dựng từ Dockerfile trong agent_service/, dựa trên Python 3.12-slim. Dockerfile cài đặt LangGraph, sentence-transformers, httpx và các thư viện cần thiết. Biến môi trường PYTHONPATH được thiết lập để Agent Service có thể nạp các mô-đun từ backend/ (models, database).

Điểm đặc biệt: healthcheck có start_period: 30s để cho phép tải mô hình BGE-M3 (~2GB) trước khi nhận request.

Các biến môi trường cấu hình chính của Agent Service:

- AGENT_ROUTER_MODE: rule | llm | hybrid – cơ chế định tuyến.
- AGENT_QUERY_REWRITE_ENABLED: Bật/tắt LLM query rewriting.
- AGENT_SPECIALIST_LLM_ENABLED: Bật/tắt LLM specialist agents.
- AGENT_REACT_ENABLED: Bật/tắt cơ chế ReAct.
- AGENT_REACT_MAX_ITERATIONS: Số vòng lặp ReAct tối đa (mặc định 2).
- AGENT_REQUEST_TIMEOUT_SECONDS: Timeout cho request (mặc định 45s).
- AGENT_LLM_TIMEOUT_SECONDS: Timeout cho LLM call (mặc định 30s).
- AGENT_LLM_MONTHLY_BUDGET_USD: Ngân sách LLM hàng tháng (mặc định \$100).
- AGENT_LLM_COST_TRACKING_ENABLED: Bật/tắt theo dõi chi phí.

Backend API

backend:

build:

context: ./backend

dockerfile: Dockerfile

container_name: realestate_backend

depends_on:

postgres:

condition: service_healthy

redis:

condition: service_healthy

agent-service:

condition: service_healthy

env_file: .env

environment:

DATABASE_URL: postgresql+asyncpg://admin:...@postgres:5432/realestate

REDIS_URL: redis://redis:6379/0

AGENT_SERVICE_URL: http://agent-service:8100

AGENT_INTERNAL_KEY: \${AGENT_INTERNAL_KEY}

CHATBOT_AGENT_SERVICE_ENABLED: \${CHATBOT_AGENT_SERVICE_ENABLED:-}

```
volumes:
  - ./data:/app/data
```

Backend API chỉ khởi động sau khi PostgreSQL, Redis và Agent Service đều đã sẵn sàng. Biến `CHATBOT_AGENT_SERVICE_ENABLED` cho phép vận hành linh hoạt: khi bật, Backend chuyển yêu cầu chatbot sang Agent Service; khi tắt, sử dụng pipeline nội bộ. Volume `./data:/app/data` cho phép Backend truy cập dữ liệu CSV.

Pipeline Worker

```
pipeline-worker:
  build:
    context: .
    dockerfile: pipeline_worker/Dockerfile
  container_name: realestate_pipeline_worker
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  env_file: .env
  environment:
    DATABASE_URL: postgresql+asyncpg://admin:...@postgres:5432/realestate
    REDIS_URL: redis://redis:6379/0
    AGENT_INTERNAL_KEY: ${AGENT_INTERNAL_KEY}
    GEMINI_API_KEY: ${GEMINI_API_KEY:-}
    HF_EMBEDDING_MODEL: BAAI/bge-m3
  volumes:
    - ./data:/app/data
```

Pipeline Worker là dịch vụ chuyên trách các tác vụ xử lý dữ liệu nặng, tách biệt khỏi Backend và Agent Service. Worker chạy các module Python (crawler, data_pipeline) dưới dạng subprocess, đảm bảo isolation về tài nguyên. Volume `./data:/app/data` cho phép Worker đọc/ghi dữ liệu CSV và tài liệu pháp lý.

Frontend

```
frontend:
```

```

build:
  context: ./frontend
  dockerfile: Dockerfile
  args:
    INTERNAL_API_URL: http://backend:8000
    NEXT_PUBLIC_API_URL: /api/v1
  container_name: realestate_frontend
ports:
  - "3000:3000"
depends_on:
  - backend

```

Frontend được xây dựng từ Node.js 22 Alpine. Quá trình build: `npm ci` → `npm run build`. Chạy ở chế độ production (`npm run start`) trên cổng 3000. Hai tham số build: `INTERNAL_API_URL` (địa chỉ nội bộ cho SSR) và `NEXT_PUBLIC_API_URL` (địa chỉ công khai cho browser).

3.1.4. Mạng nội bộ và giao tiếp giữa các dịch vụ

Các dịch vụ giao tiếp qua tên dịch vụ (DNS nội bộ của Docker):

- **Frontend** → **Backend**: `http://backend:8000` (SSR) và `/api/v1` (browser, qua Next.js proxy).
- **Backend** → **Agent Service**: `http://agent-service:8100/internal/agent/chat` với header `X-Internal-Agent-Key`.
- **Backend** → **Pipeline Worker**: `http://pipeline-worker:8200/internal/pipeline` với internal key.
- **Tất cả dịch vụ** → **PostgreSQL**: `postgresql+asyncpg://admin:...@postgres:5432`
- **Tất cả dịch vụ** → **Redis**: `redis://redis:6379/0`.

3.1.5. Quy trình triển khai

Chuẩn bị môi trường

Trước khi triển khai, cần chuẩn bị file `.env` với các biến môi trường bắt buộc:

```

# Database
POSTGRES_DB=realestate

```

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=<mat_khau_manh>
DATABASE_URL=postgresql+asyncpg://admin:<password>@postgres:5432/rea

# Redis
REDIS_URL=redis://redis:6379/0

# JWT
JWT_SECRET_KEY=<khoa_bi_mat_jwt>

# CORS
CORS_ORIGINS=http://localhost:3000

# AI/ML
GEMINI_API_KEY=<google_gemini_api_key>
COHERE_API_KEY=<cohere_api_key> # Optional
HF_EMBEDDING_MODEL=BAAI/bge-m3

# Agent Service
AGENT_INTERNAL_KEY=<internal_key>
AGENT_ROUTER_MODE=hybrid
AGENT_SPECIALIST_LLM_ENABLED=true
AGENT_QUERY_REWRITE_ENABLED=true
AGENT_REACT_ENABLED=true
CHATBOT_AGENT_SERVICE_ENABLED=true

# Frontend
NEXT_PUBLIC_API_URL=/api/v1
```

Xây dựng và khởi động

1. Xây dựng ảnh:

```
docker-compose build
```

Xây dựng ảnh Docker cho Backend, Agent Service, Pipeline Worker và Frontend.

2. Khởi động toàn bộ hệ thống:

```
docker-compose up -d
```

Docker Compose khởi động các dịch vụ theo thứ tự: PostgreSQL, Redis → Pipeline Worker, Agent Service → Backend → Frontend.

3. Khởi động riêng hạ tầng (cho phát triển):

```
docker-compose up -d postgres redis
```

Khi phát triển, chỉ cần chạy PostgreSQL và Redis qua Docker, còn các service khác chạy trực tiếp: `uvicorn app.main:app -reload -port 8000` (Backend), `uvicorn agent_service.main:app -reload -port 8100` (Agent Service).

4. Xem nhật ký:

```
docker-compose logs -f agent-service
```

5. Dừng hệ thống:

```
docker-compose down
```

Kiểm tra trạng thái triển khai

Sau khi khởi động, kiểm tra qua các health endpoints:

- **Backend API:** GET `http://localhost:8000/health` – trả về trạng thái database và Redis.
- **Agent Service:** GET `http://localhost:8100/internal/agent/health` – yêu cầu internal key, trả về service name, graph_version, llm_cost.
- **Agent Readiness:** GET `http://localhost:8100/internal/agent/readiness` – trạng thái sẵn sàng của 4 nguồn dữ liệu.
- **Pipeline Worker:** GET `http://localhost:8200/internal/pipeline/health`.
- **Frontend:** Mở `http://localhost:3000`.
- **Swagger Docs:** `http://localhost:8000/docs` – tài liệu API tương tác.

Nạp dữ liệu ban đầu

Sau khi hệ thống khởi động, nạp dữ liệu vào PostgreSQL:

```
python -m data_pipeline.load_db
```

Hoặc gọi Pipeline Worker API:

```
curl -X POST http://localhost:8200/internal/pipeline/ingest/listings \
-H "X-Internal-Agent-Key: <key>" \
-H "Content-Type: application/json" \
-d '{"csv_path": "data/apartments_cleaned.csv", "batch_size": 50}'
```

3.2. Hệ thống giám sát và quan sát

3.2.1. Observability – Theo dõi vết xử lý tác tử

Hệ thống tích hợp cơ chế observability toàn diện cho phép theo dõi toàn bộ luồng xử lý của mỗi request chatbot. Mỗi request được ghi nhận qua ba bảng chính:

- **AgentTrace:** Lưu thông tin tổng quan của một request: request_id, session_id, user_id, intent, agents_used, trace_summary (số steps, latency tổng, warnings), full_trace (JSONB – toàn bộ state cuối cùng), status (success/partial/error), graph_version, prompt_version, model_name, created_at.
- **AgentTraceStep:** Lưu chi tiết từng bước trong đồ thị: trace_id (FK đến AgentTrace), step_name (context_builder, router, specialist_agents,...), status, latency_ms, input/output (JSONB), error message nếu có.
- **AgentLLMCall:** Log mỗi lần gọi Gemini: trace_id, step_name, node_name, model_name, latency_ms, input_tokens, output_tokens, estimated_cost_usd, skipped_reason (nếu không gọi được).

Ngoài ra còn có **AgentRetrievalEvent** ghi lại mỗi sự kiện truy xuất: tool_name, domain, filters, result_count, latency_ms.

3.2.2. Đánh giá chất lượng tự động (LLM Judge)

Hệ thống tích hợp cơ chế đánh giá chất lượng phản hồi tự động qua endpoint POST /internal/agent/evaluate. Judge sử dụng Gemini 2.0 Flash với prompt tiếng Việt để chấm điểm phản hồi theo 5 metrics:

1. **Groundedness (Tính căn cứ):** Phản hồi có được hỗ trợ bởi các nguồn tham khảo hay không.
2. **Helpfulness (Tính hữu ích):** Phản hồi có trực tiếp giúp ích cho người dùng bất động sản hay không.
3. **Citation Quality (Chất lượng trích dẫn):** Nguồn tham khảo có liên quan, rõ ràng và đúng với nội dung trả lời hay không.
4. **Safety (An toàn):** Tránh khẳng định pháp lý/tài chính tuyệt đối, thông tin nhạy cảm hoặc nội dung nguy hiểm.
5. **Trace Completeness (Tính đầy đủ của vết xử lý):** Trace có thể hiện đủ agent, bước RAG, cảnh báo và thông tin quan sát cần thiết hay không.

Mỗi metric được chấm điểm từ 0.0 đến 1.0 kèm rationale giải thích. Kết quả được lưu vào bảng **EvalRun** để phân tích và cải thiện hệ thống. Ngoài ra, hệ thống còn có cơ chế cleanup tự động: các eval runs đang chạy quá 30 phút sẽ được đánh dấu failed.

3.2.3. Theo dõi chi phí LLM

Để kiểm soát chi phí sử dụng Gemini API, hệ thống tích hợp cơ chế cost tracking qua module `agent_service/llm/cost.py`:

- Mỗi lần gọi Gemini được ghi nhận: `input_tokens`, `output_tokens`, `estimated_cost_usd`.
- Trước mỗi lần gọi, hệ thống kiểm tra `AGENT_LLM_MONTHLY_BUDGET_USD`.
- Nếu vượt ngân sách, LLM bị vô hiệu hóa và hệ thống chuyển sang deterministic mode.
- Cost summary được hiển thị trong health check endpoint.

3.2.4. Prometheus + Grafana + Alertmanager

Hệ thống giám sát được triển khai qua bộ ba Prometheus (thu thập metrics), Grafana (trực quan hóa) và Alertmanager (cảnh báo):

- **Prometheus:** Scrape các metrics endpoint từ Backend, Agent Service và Pipeline Worker. Các metrics chính: request count, latency histogram, error rate, pipeline run status.
- **Grafana:** Dashboard hiển thị: số lượng chat request theo thời gian, latency phân bố, tỷ lệ lỗi, trạng thái pipeline, chi phí LLM.

- **Alertmanager:** Cảnh báo khi: pipeline thất bại 2 lần liên tiếp, latency vượt ngưỡng, error rate > 5%, LLM budget còn dưới 10%.

3.2.5. Admin Dashboard

Backend cung cấp admin dashboard qua các API endpoints:

- GET /api/v1/admin/pipeline-readiness – Trạng thái sẵn sàng của 4 nguồn dữ liệu (listings, projects, news, legal).
- GET /api/v1/admin/agent-traces – Danh sách AgentTrace với phân trang, lọc theo status và thời gian.
- GET /api/v1/admin/agent-traces/{id} – Chi tiết một trace với đầy đủ steps, LLM calls, retrieval events.
- GET /api/v1/admin/chat-feedback – Phản hồi người dùng.
- GET /api/v1/admin/memory-proposals – Đề xuất ghi nhớ đang chờ xác nhận.

3.3. Kiểm thử hệ thống

3.3.1. Chiến lược kiểm thử

Hệ thống được kiểm thử theo chiến lược đa tầng, từ thấp đến cao:

1. **Kiểm thử đơn vị (Unit Testing):** Kiểm tra từng hàm, lớp, mô-đun độc lập, sử dụng pytest với mock objects để cô lập phụ thuộc ngoại vi (cơ sở dữ liệu, API, hệ thống tệp). Đây là tầng có số lượng ca kiểm thử lớn nhất và chạy nhanh nhất.
2. **Kiểm thử tích hợp (Integration Testing):** Kiểm tra tương tác giữa các mô-đun với nhau và với các dịch vụ bên ngoài (PostgreSQL, Redis, Gemini API), sử dụng cơ sở dữ liệu kiểm thử hoặc service mocks.
3. **Kiểm thử hệ thống (System Testing):** Kiểm tra toàn bộ luồng đầu cuối, từ frontend đến backend đến database, đảm bảo các chức năng hoạt động đúng như mong đợi.
4. **Kiểm thử chấp nhận (Acceptance Testing):** Kiểm tra hệ thống đáp ứng yêu cầu nghiệp vụ đã đề ra, thực hiện thủ công với các kịch bản thực tế.
5. **Kiểm thử hiệu năng (Performance Testing):** Đo lường thời gian phản hồi, throughput và mức tiêu thụ tài nguyên của hệ thống dưới các mức tải khác nhau, đảm bảo đáp ứng các yêu cầu phi chức năng đã đề ra.

3.3.2. Công cụ và khung công tác

- **pytest:** Khung công tác kiểm thử chính cho toàn bộ mã nguồn Python (backend, agent_service, data_pipeline). Sử dụng plugin: pytest-asyncio (hỗ trợ kiểm thử bất đồng bộ), pytest-cov (đo độ bao phủ), pytest-benchmark (đo hiệu năng vi mô).
- **ESLint:** Phân tích tĩnh cho frontend TypeScript, phát hiện lỗi cú pháp và vấn đề về phong cách mã nguồn.
- **compileall:** Mô-đun chuẩn của Python dùng để kiểm tra cú pháp toàn bộ tệp mã nguồn, phát hiện lỗi nạp thư viện mà không cần chạy kiểm thử.
- **unittest.mock + pytest.monkeypatch:** Tạo mock objects cho dịch vụ bên ngoài.
- **locust:** Công cụ kiểm thử tải (load testing) cho REST API, cho phép mô phỏng hàng trăm người dùng đồng thời và đo lường thời gian phản hồi.

3.3.3. Kiểm thử chức năng theo phân tích yêu cầu

Dựa trên các nhóm chức năng đã phân tích trong Chương 2, em thiết kế các ca kiểm thử chính như bảng 3.1. Mỗi nhóm chức năng được kiểm thử ở cả ba mức: đơn vị, tích hợp và hệ thống.

Nhóm chức năng	Ca kiểm thử chính	Tập kiểm thử
Xác thực & tài khoản	• Đăng ký với email hợp lệ/thùng rác • Đăng nhập sai mật khẩu • JWT token hết hạn/tampered • Phân quyền admin/superuser	Kiểm thử tích hợp auth router

Duyệt & tìm kiếm BDS	• Lọc theo loại tin (bán/thuê) • Lọc đa tiêu chí (city, district, price, area, bedrooms) • Phân trang & sắp xếp • Tìm kiếm full-text • Kết quả rỗng với bộ lọc quá hẹp	test_listings_route_order.py
test_hybrid_search.py test_backend_hybrid_search.py		
Chi tiết BDS	• Hiện thị đủ 30 trường • Gợi ý BDS tương tự • BDS không tồn tại → 404	test_public_content_routes.py
Thống kê thị trường	• API market stats có dữ liệu • Giá theo quận (price-by-district) • Phân bố loại hình (property-type-count) • Lọc theo thành phố	test_market_stats.py
test_market_snapshot_ingestor.py		

Chatbot đa tác tử	• Router phân loại đúng intent (6 loại) • Router chọn đúng agent mục tiêu • Multi-agent chạy song song • Phản hồi có trích dẫn nguồn • Fallback khi không có dữ liệu • Kiểm tra safety disclaimer • Giới hạn quota (20/200 lượt/ngày) • Chặn abuse (spam)	test_agent_graph_core.py
test_router_modes.py test_specialists_parallel.py test_chat_abuse_guard.py test_chat_quota.py test_production_chatbot.py		
Investment Model	• Tính ROI, NPV, dòng tiền • Giả định mặc định & tùy chỉnh • Thiếu evidence → missing_evidence	test_investment_model.py
test_investment_metrics.py		

test_investment_calculators.py		
Committee Review	• Ba góc nhìn (Bull/Bear/Skeptic) • Mỗi góc nhìn có stance, risk_level • Evidence IDs hợp lệ	test_committee_review.py
Blackboard System	• Ghi entry từ agent • Đọc entry từ agent khác • Validation evidence IDs • Deduplication entries	test_agent_blackboard.py
test_blackboard_specialists.py		
ReAct Controller	• Thiếu evidence → retrieve_more • Sau retrieve → chạy lại specialist • Giới hạn 2 vòng lặp • Fallback khi hết vòng lặp	test_react_loop.py

Memory System	• Extract district từ query • Tạo MemoryProposal • User confirm/reject proposal • Ưu tiên dùng precomputed tags	test_memory_extraction.py
		test_memory_preferences.py test_chunk_uses_precomputed_tags.py
Data Pipeline	• Clean: chuẩn hóa giá, diện tích • Chunk: 4 loại chunk/listing • Embed: BGE-M3 1024d • Load: upsert PostgreSQL • Ingestors: listings, projects, news	test_clean.py
		test_chunk.py test_embed.py test_listings_ingestor.py test_projects_ingestor.py test_news_ingestor.py

Legal KB	• Parse HTML & PDF • Chunk theo Điều/Khoản • Citation đầy đủ • Manifest & slug	test_legal_chunker.py
test_legal_html_parser.py test_legal_pdf_parser.py test_legal_structure.py test_legal_manifest.py		
Search & Retrieval	• Hybrid search (SQL + vector) • Reranking (Cohere) • Redis caching • Retrieval planner đa nguồn	test_hybrid_search.py
test_hybrid_search_caching.py test_hybrid_search_multi_parent.py test_retrieval_parallel.py		
Observability & Admin	• AgentTrace ghi đúng • AgentTraceStep đủ bước • Admin API auth • Metrics endpoint	test_agent_observability_pers
test_admin_observability.py test_metrics_endpoint.py		

Pipeline & Crawler	• Crawler core (CSV, parser) • Pipeline runner • DAG structure • Pipeline Worker health	test_crawler_core.py
test_pipeline_runner.py test_dag_structure.py test_pipeline_worker.py		

Bảng 3.1: Các ca kiểm thử chính theo nhóm chức năng

3.3.4. Kiểm thử hiệu năng

Bên cạnh kiểm thử chức năng, em tiến hành kiểm thử hiệu năng ở hai cấp độ: vi mô (micro-benchmark) cho các hàm và mô-đun riêng lẻ, và vĩ mô (load testing) cho toàn bộ hệ thống API.

Kiểm thử hiệu năng vi mô (Micro-benchmark)

Sử dụng `pytest-benchmark` để đo lường thời gian thực thi của các thao tác then chốt trong hệ thống:

- **Embedding (BGE-M3):** Đo thời gian encode một batch 16 văn bản. Mục tiêu: < 2s/batch trên CPU, đảm bảo pipeline indexing hoàn thành trong thời gian hợp lý cho 100.000+ listings.
- **Hybrid Search:** Đo thời gian truy vấn PostgreSQL kết hợp WHERE filter và HNSW vector search. Mục tiêu: < 500ms cho truy vấn cơ bản, < 1000ms cho truy vấn phức tạp với JOIN và ORDER BY cosine distance.
- **Chunking:** Đo thời gian `build_listing_chunks()` cho 10.000 listings. Mục tiêu: < 5s, đảm bảo pipeline chunking không trở thành bottleneck.
- **Redis Cache:** Đo hit rate và latency của embedding cache và search result cache. Mục tiêu: hit rate > 60% cho truy vấn lặp, cache lookup < 5ms.

- **LangGraph Node:** Đo thời gian từng node trong đồ thị (context_builder, router, retrieval_planner, specialist_agents, synthesizer) qua AgentTraceStep.latency_ms. Mục tiêu: mỗi node < 2s, tổng thời gian đồ thị < 10s cho chế độ deterministic, < 45s cho chế độ LLM.

Kiểm thử tải (Load Testing)

Sử dụng `locust` để mô phỏng nhiều người dùng đồng thời truy cập hệ thống, tập trung vào các API endpoint chính:

- **Kịch bản 1 – Duyệt listings (đọc):** 50 users đồng thời gọi GET `/api/v1/listings` với bộ lọc cơ bản (city, district, listing_type). Mục tiêu: p95 latency < 500ms, 0 lỗi.
- **Kịch bản 2 – Chatbot (agent pipeline):** 10 users đồng thời gọi POST `/api/v1/chat` với truy vấn đơn giản (“tìm căn hộ Quận 7”). Mục tiêu: p95 latency < 10s, tỷ lệ timeout < 5%.
- **Kịch bản 3 – Hybrid search (đọc + vector):** 20 users đồng thời gọi GET `/api/v1/listings` với full-text search + bộ lọc. Mục tiêu: p95 latency < 1000ms.
- **Kịch bản 4 – Hỗn hợp thực tế:** Mô phỏng traffic thực tế với 80% đọc listings, 15% chatbot, 5% market stats. 100 users đồng thời trong 5 phút. Mục tiêu: p95 latency < 5s tổng thể, error rate < 1%.

Kết quả kiểm thử tải được ghi nhận qua Prometheus metrics (request_count, request_duration_seconds) và hiển thị trên Grafana dashboard, cho phép em xác định bottleneck và tối ưu hệ thống trước khi triển khai.

Kết quả kiểm thử hiệu năng dự kiến

Dựa trên kiến trúc hệ thống và kết quả kiểm thử đơn vị, em kỳ vọng hệ thống đáp ứng các chỉ tiêu sau:

Chỉ tiêu	Mục tiêu	Đo lường qua
API listings (bộ lọc cơ bản)	< 500ms	locust + Prometheus histogram
API listings (hybrid search)	< 1000ms	locust + Prometheus histogram
Chatbot (deterministic, 1 agent)	< 5s	AgentTraceStep.latency_ms

Chatbot (LLM, multi-agent)	< 45s	AgentTraceStep.latency_ms
Chatbot (ReAct, 1 vòng lặp)	< 60s	AgentTraceStep.latency_ms
Embedding batch (16 texts)	< 2s	pytest-benchmark
Chunking 10.000 listings	< 5s	pytest-benchmark
Redis cache hit rate	> 60%	Redis INFO stats
Throughput (100 users hỗn hợp)	> 50 req/s	locust + Prometheus
Error rate (load test)	< 1%	locust + Prometheus

Bảng 3.2: Chỉ tiêu hiệu năng dự kiến

3.3.5. Kết quả kiểm thử

Hệ thống có hơn 80 tập kiểm thử bao phủ toàn bộ các mô-đun chính:

- **Backend API (35+ tập):** Kiểm thử các routers (listings, market, chat, auth, projects, articles, preferences, admin), services (chatbot pipeline, RAG, agent_service client, observability), và models.
- **Agent Service (20+ tập):** Kiểm thử graph workflow (smoke test end-to-end và từng node), router (rule/LLM/hybrid modes), query understanding, retrieval planner (đa nguồn, song song), specialist agents (deterministic + LLM modes, song song), blackboard system, investment model (calculators, metrics, safety), committee review, synthesis, safety validator, react controller (vòng lặp, timeout), memory extraction, conversation context, và readiness cache.
- **Data Pipeline (15+ tập):** Kiểm thử clean, chunk (listing, legal), embed, enrich (geocode, intent), load_db, và các ingestors (listings, projects, news, legal KB).
- **Crawler (5+ tập):** Kiểm thử core (csv_writer, parser), sale/rent parsers, news parsers, project parsers.
- **Frontend:** ESLint passing, kiểm thử API client và components.
- **Infrastructure (10+ tập):** Kiểm thử Docker config (agent_service, pipeline_worker, airflow), pipeline runner, DAG structure, metrics endpoint, alerting config.

3.3.6. Các vấn đề đã phát hiện và hướng khắc phục

Trong quá trình kiểm thử, các vấn đề chính được phát hiện và khắc phục:

- **LLM features bị tắt mặc định:** Router và specialist agents mặc định dùng rule-based/deterministic mode. Cần bật `AGENT_ROUTER_MODE=hybrid` và `AGENT_SPECIALIS` trước khi public demo. Đã có đầy đủ code LLM router và LLM specialist wrapper, chỉ cần bật flag.
- **Memory system còn sơ khai:** Chỉ extract được district từ query qua keyword matching. Cần tích hợp LLM-based memory extraction để hỗ trợ đầy đủ city, budget, property_type, bedrooms. Đã có sẵn infrastructure (MemoryProposal model, user confirm/reject API).
- **Thiếu kiểm thử hiệu năng:** Hệ thống chưa có bộ kiểm thử tải (load testing) và kiểm thử hiệu năng vi mô (micro-benchmark). Cần bổ sung locust cho load testing và pytest-benchmark cho micro-benchmark trước khi triển khai thực tế.
- **Secret keys mặc định:** `JWT_SECRET_KEY` và `AGENT_INTERNAL_KEY` có giá trị mặc định trong code và docker-compose.yml, cần override qua `.env` khi triển khai thực tế.

KẾT LUẬN

Kết quả đạt được

Sau quá trình nghiên cứu và triển khai, đồ án đã đạt được các kết quả chính sau đây.

Về xây dựng nền tảng: Đồ án đã xây dựng thành công một nền tảng web bất động sản hoàn chỉnh với giao diện người dùng hiện đại sử dụng Next.js 16 và React 19, máy chủ dịch vụ FastAPI. Nền tảng cung cấp đầy đủ các chức năng cho người dùng cuối: duyệt danh sách nhà đất bán và cho thuê, lọc đa tiêu chí (loại hình, khu vực, giá, diện tích, số phòng, hướng nhà), sắp xếp theo giá và diện tích, xem chi tiết bất động sản với ba mươi trường thông tin, bảng thống kê thị trường trực quan, cùng hệ thống xác thực người dùng qua mã thông báo và bảng điều khiển quản trị.

Về chatbot đa tác tử: Đồ án đã phát triển thành công chatbot đa tác tử với sáu tác tử chuyên biệt đảm nhận các vai trò tìm kiếm bất động sản, tra cứu dự án, tra cứu tin tức, phân tích thị trường, tư vấn pháp lý và tư vấn đầu tư. Chatbot có khả năng hiểu ý định người dùng bằng tiếng Việt qua ba cơ chế định tuyến (dựa trên từ khóa, mô hình ngôn ngữ lớn và kết hợp), điều phối đến tác tử phù hợp, truy xuất dữ liệu qua tìm kiếm lai (kết hợp truy vấn có cấu trúc, tìm kiếm véc-tơ qua chỉ mục HNSW, xếp hạng lại và bộ nhớ đệm Redis) và sinh phản hồi có trích dẫn nguồn rõ ràng. Hệ thống tích hợp mô hình phân tích đầu tư với các chỉ số tài chính (ROI, NPV, dòng tiền), cơ chế đánh giá hội đồng đa góc nhìn (bull/bear/skeptic), và vòng lặp ReAct cho phép truy xuất bổ sung khi thiếu bằng chứng.

Về kiến trúc hệ thống: Đồ án đã triển khai kiến trúc chatbot hai lớp với đường ống nội bộ xử lý nhanh cho người dùng cuối và Dịch vụ Tác tử chạy đồ thị trạng thái LangGraph 14 nút riêng biệt cho suy luận đa bước nâng cao. Đồ thị bao gồm đầy đủ các pha: hiểu (context builder, readiness checker, router), truy xuất (query understanding, retrieval planner), suy luận (6 specialist agents song song, investment model, committee review), và phản hồi (synthesizer, safety validator, ReAct controller, memory proposals). Hệ thống tích hợp Bảng đen liên tác tử (Agent Blackboard) cho phép các agent chia sẻ kết quả phân tích, cơ chế ghi nhớ và sở thích người dùng để cá nhân hóa phản hồi, và hệ thống đánh giá chất lượng tự động qua năm tiêu chí. Toàn bộ hệ thống được thiết kế theo kiến trúc dịch vụ vi mô với sáu dịch vụ độc lập trong Docker Compose, có khả năng mở rộng và triển khai dễ dàng trên nhiều môi trường.

Về đường ống dữ liệu: Đồ án đã xây dựng đường ống thu thập và xử lý dữ liệu tự động hoàn chỉnh từ batdongsan.com.vn. Quy trình bao gồm: thu thập dữ liệu bằng công cụ tự động hóa trình duyệt với cơ chế ẩn danh cho cả tin bán, tin cho thuê, dự án và tin tức;

làm sạch và chuẩn hóa dữ liệu; làm giàu (mã hóa địa lý qua dịch vụ bản đồ, gán nhãn nhu cầu); chia đoạn ngữ nghĩa với bốn loại đoạn cho mỗi bất động sản; tạo véc-tơ đặc trưng 1024 chiều bằng mô hình BGE-M3; và nạp vào PostgreSQL với tìm kiếm véc-tơ qua các mô-đun chuyên biệt. Đường ống được lập lịch tự động qua Apache Airflow với bốn luồng công việc.

Về tìm kiếm: Đồ án đã triển khai hệ thống tìm kiếm lai hiệu quả, kết hợp giữa lọc có cấu trúc cho dữ liệu quan hệ và tìm kiếm véc-tơ với chỉ mục HNSW cho tìm kiếm ngữ nghĩa, có xếp hạng lại qua mô hình chuyên dụng đa ngôn ngữ và bộ nhớ đệm qua Redis để cải thiện độ chính xác và tốc độ.

Về giám sát và chất lượng: Đồ án đã xây dựng hệ thống quan sát toàn diện với theo dõi vết tác tử (AgentTrace, AgentTraceStep, AgentLLMCall, AgentRetrievalEvent), đánh giá chất lượng tự động qua LLM Judge với năm tiêu chí (tính căn cứ, tính hữu ích, chất lượng trích dẫn, an toàn, tính đầy đủ của vết xử lý), theo dõi chi phí LLM theo ngân sách hàng tháng, giám sát trạng thái nguồn dữ liệu (Readiness Snapshot), và chỉ số hệ thống qua Prometheus + Grafana + Alertmanager. Bộ kiểm thử với hơn 60 tệp bao phủ toàn bộ các mô-đun: đường ống chatbot, dịch vụ tác tử (graph workflow, router, specialists, blackboard, committee, ReAct), tìm kiếm lai, tạo sinh tăng cường truy xuất, đường ống dữ liệu, kho kiến thức pháp lý, thu thập dữ liệu, ghi nhớ/sở thích và quản trị.

Hạn chế

Bên cạnh những kết quả đạt được, đồ án vẫn còn một số hạn chế cần được khắc phục trong các phiên bản tiếp theo.

Thứ nhất, về nguồn dữ liệu, hệ thống hiện chỉ thu thập dữ liệu từ một nguồn duy nhất là batdongsan.com.vn. Điều này hạn chế độ phủ của dữ liệu, đặc biệt ở các khu vực mà nền tảng này chưa chiếm ưu thế. Việc tích hợp thêm các nguồn dữ liệu khác như alonhadat.com.vn, chotot.com sẽ giúp tăng đáng kể số lượng và chất lượng tin đăng.

Thứ hai, về chất lượng véc-tơ đặc trưng tiếng Việt, mô hình BGE-M3 tuy hỗ trợ tiếng Việt nhưng chưa được tối ưu riêng cho ngôn ngữ này. Các mô hình véc-tơ chuyên biệt cho tiếng Việt như PhoBERT, ViT5 cần được đánh giá và thử nghiệm để cải thiện độ chính xác của tìm kiếm ngữ nghĩa.

Thứ ba, về khả năng hội thoại của chatbot, hệ thống hiện tại vẫn gặp khó khăn với các câu hỏi phức tạp đòi hỏi suy luận qua nhiều bước hoặc tổng hợp thông tin từ nhiều nguồn khác nhau. Cơ chế ghi nhớ người dùng còn đơn giản, chưa hỗ trợ ghi nhớ ngữ cảnh dài hạn qua nhiều phiên.

Thứ tư, về giao diện người dùng, bảng điều khiển quản trị và các trang thống kê thị

trường còn ở mức cơ bản, chưa có nhiều biểu đồ tương tác và khả năng lọc sâu. Trải nghiệm trên thiết bị di động cần được cải thiện thêm.

Thứ năm, về vận hành, hệ thống chưa được triển khai thực tế trên môi trường sản xuất với tên miền công cộng, chứng chỉ bảo mật và cân bằng tải. Khả năng chịu lỗi và phục hồi của hệ thống chưa được kiểm thử đầy đủ dưới tải cao.

Thứ sáu, về kiểm thử, mặc dù đã có hơn 60 tệp kiểm thử nhưng độ bao phủ mã nguồn còn chưa đồng đều giữa các mô-đun. Các bài kiểm thử đầu cuối cho luồng chatbot hoàn chỉnh còn hạn chế do phụ thuộc vào API bên ngoài.

Hướng phát triển

Từ những hạn chế đã nêu, đề án đề xuất các hướng phát triển trong tương lai.

Về dữ liệu, cần mở rộng thu thập từ nhiều nguồn bất động sản khác nhau, bổ sung dữ liệu về quy hoạch đô thị, hạ tầng giao thông và tiện ích xung quanh (trường học, bệnh viện, trung tâm thương mại) để làm giàu thông tin cho mỗi bất động sản.

Về chatbot, cần nâng cấp lên các mô hình ngôn ngữ mới hơn, hỗ trợ suy luận đa bước tốt hơn và xử lý ngữ cảnh dài. Tích hợp thêm tác tử chuyên biệt mới như tác tử so sánh bất động sản, tác tử dự báo giá, tác tử tư vấn vay ngân hàng. Cải thiện cơ chế ghi nhớ để hỗ trợ ghi nhớ dài hạn và cá nhân hóa sâu hơn. Bổ sung khả năng đa phương thức: cho phép người dùng tải lên hình ảnh bất động sản để chatbot phân tích và so sánh.

Về tìm kiếm, cần thử nghiệm các mô hình véc-tơ tiếng Việt chuyên biệt như PhoBERT để cải thiện chất lượng tìm kiếm ngữ nghĩa. Nghiên cứu áp dụng tìm kiếm lai kết hợp tìm kiếm toàn văn bản, tìm kiếm véc-tơ và tìm kiếm dựa trên đồ thị tri thức.

Về nền tảng, cần bổ sung các chức năng mới như so sánh bất động sản cạnh nhau, bản đồ nhiệt giá, biểu đồ xu hướng giá theo thời gian, cảnh báo giá khi có bất động sản phù hợp với tiêu chí người dùng, và xây dựng cộng đồng người dùng với tính năng bình luận và đánh giá.

Về triển khai, cần chuyển đổi từ Docker Compose sang Kubernetes để vận hành ở quy mô sản xuất với khả năng tự động mở rộng, cân bằng tải và phục hồi. Thiết lập đường ống tích hợp liên tục và triển khai liên tục để tự động hóa kiểm thử và phát hành. Bổ sung giám sát nâng cao với Grafana để trực quan hóa chỉ số hệ thống.

Về kiểm thử, cần tăng độ bao phủ kiểm thử, đặc biệt là kiểm thử đầu cuối cho các luồng nghiệp vụ chính. Xây dựng bộ kiểm thử hiệu năng để đảm bảo hệ thống đáp ứng được tải người dùng trong thực tế.

PHỤ LỤC

Phụ lục A CẤU TRÚC THƯ MỤC DỰ ÁN

Cấu trúc thư mục chính của dự án (rút gọn):

```
1 RealEstate_Chatbot_v2/|—
2   frontend/                                     # Next.js 16 + React 19 + TypeScript|—
3     app/|||—
4       page.tsx                                # Trang ùch||—
5       layout.tsx                             # Root layout||—
6       nha-dat-ban/                           # Danh sách & chi ếtit ĐBS bán|||—
7         page.tsx|||—
8         [id]/                                # Trang chi ếtit (dynamic route)||—
9       nha-dat-cho-thue/                       # Danh sách & chi ếtit cho thuê||—
10      thi-truong/                             # Dashboard ồthng kê ịth uờtrng||—
11      admin/                                  # àBng ềđiêu ềkhin àqun ịtr||—
12      dang-nhap/                              # Đăng ậnhp||—
13      dang-ky/                               # Đăng ký|—
14      components/|||—
15        layout/                               # Header.tsx, Footer.tsx||—
16        listing/                             # ListingCard.tsx, ListingGrid.tsx||—
17        search/                              # FilterPanel.tsx||—
18        chatbot/                             # ChatWidget.tsx||—
19        admin/                               # AdminDashboard.tsx|—
20      lib/|||—
21        api.ts                                # Typed API client (FastAPI)||—
22        types.ts                             # TypeScript interfaces||—
23        utils.ts                             # Format helpers||—
24        chatSourceDisplay.ts                 # Chat source rendering|—
25      public/                                 # Static assets|—
26
27      backend/                                # Backend API (FastAPI + SQLAlchemy)|—
28        app/|||—
29          main.py                             # FastAPI app, CORS, router registration||—
30          config.py                           # Settings (env vars, defaults)||—
31          database.py                         # Async engine, session, Base||—
32          models/                             # 14+ ORM models|||—
33            listing.py                        #   - listings (30+ fields)|||—
34            chunk.py                          #   - chunks (pgvector 1024d)|||—
35            chat.py                           #   - chat_sessions, chat_messages|||—
36            user.py                           #   - users|||—
37            project.py                        #   - projects|||—
38            article.py                        #   - articles|||—
39            pipeline_run.py                   #   - pipeline_runs|||—
```

```

40     preference.py      # - user_preferences,|||
41                        # memory_proposals,|||
42                        # chat_feedback|||└─
43     agent_observability.py # agent_traces,|||
44                        # agent_trace_steps,|||
45                        # agent_llm_calls,|||
46                        # agent_retrieval_events|||└─
47     source_readiness.py  # source_readiness||└─
48     schemas/            # Pydantic v2 contracts|||└─
49     listing.py, chat.py, auth.py|||└─
50     common.py, admin.py, preferences.py||└─
51     routers/           # 7 router modules|||└─
52     listings.py        # - /api/v1/listings|||└─
53     market.py         # - /api/v1/market|||└─
54     chat.py           # - /api/v1/chat|||└─
55     auth.py           # - /api/v1/auth|||└─
56     preferences.py     # - /api/v1/preferences|||└─
57     admin.py          # - /api/v1/admin||└─
58     metrics.py        # - /metrics||└─
59     services/||└─
60     chatbot/          # Pipeline multi-agent noi bo|||└─
61     router.py         # Intent routing (keyword + Gemini)|||└─
62     orchestrator.py   # Agent coordination|||└─
63     context.py        # Conversation context|||└─
64     memory.py         # User memory & preferences|||└─
65     contracts.py      # RoutingDecision, AgentResult|||└─
66     analytics.py      # Pipeline analytics||└─
67     agents/          # 4 specialist agents|||└─
68     property.py, market.py|||└─
69     legal.py, investment.py||└─
70     rag/             # RAG services|||└─
71     hybrid_search.py  # SQL + pgvector + rerank + cache|||└─
72     embeddings.py     # BGE-M3 embedder|||└─
73     simple_rag.py     # Filter extraction|||└─
74     cache.py         # Redis caching||└─
75     ingest.py        # Vector store ingestion||└─
76     agent_service/    # Agent Service client||└─
77     client.py        # HTTP client + internal key auth||└─
78     contracts.py     # Request/response schemas|└─
79     tests/           # 58 test files||└─
80     conftest.py       # Shared fixtures||└─
81     fixtures/        # Test data||└─
82     test_*.py        # Organized by module|└─
83     alembic/         # DB migrations|└─
84     scripts/         # Utility scripts|└─
85     requirements.txt |└─
86     Dockerfile|└─

```

```

87
88 agent_service/                                # Internal LangGraph Agent Service| |—
89     main.py                                    # FastAPI app (port 8100)| |—
90     config.py                                 # Agent-specific settings| |—
91     security.py                              # Internal key authentication| |—
92     contracts.py                             # AgentChatRequest/Response| |—
93     graph/| | |—
94         workflow.py                           # 8-node StateGraph| | |—
95         state.py                               # AgentGraphState| | |—
96         nodes.py                             # Node implementations| | |—
97         retrieval_planner.py                  # Multi-step retrieval planning| |—
98     agents/| | |—
99         specialists.py                        # 4 specialist agents| |—
100    tools/| | |—
101        retrieval.py                          # Hybrid search tool| | |—
102        market.py                            # Market stats tool| | |—
103        readiness.py                         # Data readiness check| |—
104    llm/| | |—
105        gemini.py                             # Gemini 2.0 Flash wrapper| |—
106    evaluation/| | |—
107        judge.py                             # LLM Judge (5 metrics)| |—
108    requirements.txt| |—
109    Dockerfile| |—
110
111    crawler/                                    # Playwright web scrapers| |—
112        core/                                  # Shared infrastructure| | |—
113            csv_writer.py                     # CSV with dedup| | |—
114            parser.py                         # Base parser| | |—
115            listing_detail_parser.py          # Full detail parser| |—
116    sale/                                       # Sale listings| | |—
117        crawl_urls.py                         # URL discovery (30 pages)| | |—
118        crawl_details.py                     # Detail scraping (4 workers)| |—
119    rent/                                       # Rental listings| | |—
120        crawl_urls.py| | |—
121        crawl_details.py| |—
122    projects/                                  # Real estate projects| | |—
123        crawl_urls.py| | |—
124        crawl_details.py| |—
125    news/                                       # News & articles| |—
126        crawl_articles.py| |—
127
128    data_pipeline/                             # ETL pipeline| |—
129        clean.py                             # Data cleaning & normalization| |—
130        enrich.py                            # Geocoding + intent tagging| |—
131        chunk.py                             # Semantic chunking (4 types)| |—
132        embed.py                             # BGE-M3 embedding (1024d, batch 16)| |—
133        load_db.py                           # PostgreSQL upsert| |—

```

```

134 ingestors/                                # Source-specific ingestors||└─
135     listings_ingestor.py||└─
136     projects_ingestor.py||└─
137     news_ingestor.py||└─
138     legal_kb_ingestor.py|└─
139 legal/                                    # Legal KB processing|└─
140     chunker.py                            # Legal document chunking|└─
141     html_parser.py                       # HTML legal docs|└─
142     pdf_parser.py                        # PDF legal docs|└─
143     structure.py                         # Document structure|└─
144     manifest.py                          # Version tracking|└─
145     slug_disambiguation.py|└─
146
147 airflow/                                # Workflow orchestration|└─
148     dags/                                # 4 scheduled DAGs||└─
149         daily_listings_dag.py            # Daily 02:00 HCM||└─
150         weekly_projects_dag.py          # Sunday 03:00||└─
151         weekly_news_dag.py              # Sunday 04:00||└─
152         monthly_legal_kb_dag.py         # 1st day 05:00|└─
153     plugins/                             # Airflow plugins||└─
154         pipeline_runner.py              # Crawl & ingest runner||└─
155         alerting.py                     # Slack + email alerts||└─
156         run_metrics.py                  # Pipeline metrics|└─
157     Dockerfile|└─
158     requirements.txt|└─
159
160 chatbot/                                # Legacy LangGraph sandbox|└─
161     graph.py, state.py, config.py|└─
162     agents/                             # Experimental agents|└─
163     tools/                              # Experimental tools|└─
164
165 docs/                                  # Documentation|└─
166     architecture.drawio                 # System architecture diagram|└─
167     implementation_plan.md|└─
168     multiagent-workflow.md|└─
169     pipeline.md|└─
170
171 report/                                # LaTeX report|└─
172     main.tex|└─
173     preamble.tex|└─
174     contents/                            # Chapter files|└─
175     figures/                             # Architecture diagram (PNG)|└─
176
177 data/                                  # Data directory|└─
178     raw/                                # Raw crawl output (CSV)|└─
179     knowledge/                          # Legal KB source files|└─
180

```

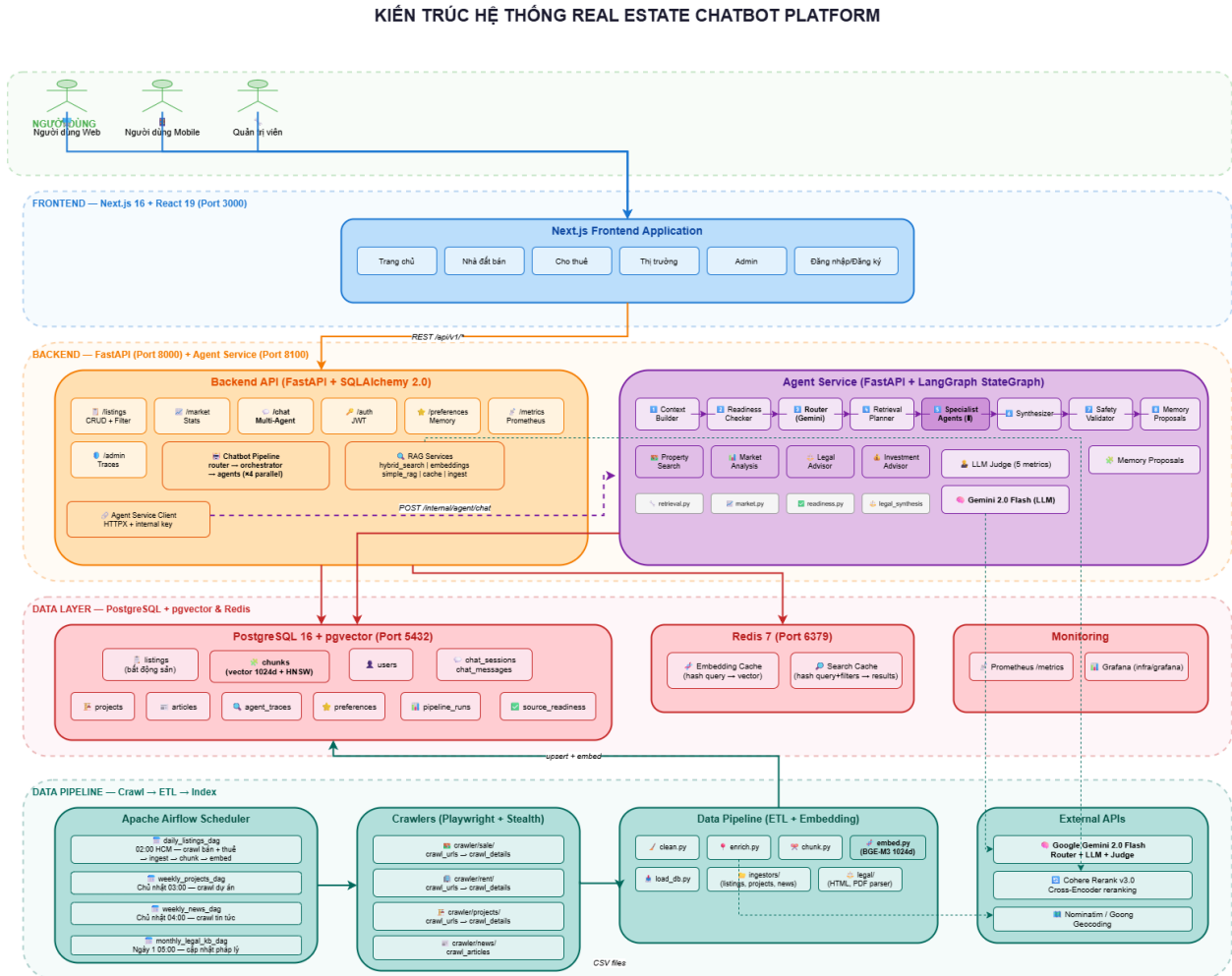
```

181 | infra/                                # Infrastructure config|└─
182 |   grafana/                            # Grafana dashboard configs|└─
183 |
184 | notebooks/                           # Jupyter notebooks (EDA)|└─
185 |
186 | docker-compose.yml                    # 5-service orchestration|└─
187 | docker-compose.override.yml└─
188 | .env                                # Environment variables

```

Phụ lục B SƠ ĐỒ KIẾN TRÚC & LUỒNG DỮ LIỆU

Sơ đồ kiến trúc tổng thể



Hình 3.2: Sơ đồ kiến trúc tổng thể hệ thống (5 tầng: Người dùng – Frontend – Backend & Agent Service – Data – Pipeline)

Luồng dữ liệu từ Crawl đến Hiển thị

- Crawl (Playwright + Stealth):** Thu thập dữ liệu từ batdongsan.com.vn với 4 workers song song. Output: file CSV UTF-8 BOM, deduplicate theo product_id. Thực hiện bởi 4 module crawler (sale, rent, projects, news).
- Clean (data_pipeline/clean.py):** Chuẩn hóa dữ liệu thô – parse giá (“4,68 tỷ” → price=4.68, unit=billion), parse diện tích (“75 m²” → area=75), phân loại property_type, chuẩn hóa địa chỉ (tách ward, district, city).
- Enrich (data_pipeline/enrich.py):** Làm giàu dữ liệu với 2 tác vụ: (a) Geocoding –

gọi Nominatim/Goong API chuyển địa chỉ text → tọa độ (latitude, longitude); (b) Intent Tagging – phân tích mô tả, gán nhãn nhu cầu (“gần trường học”, “view đẹp”, “an ninh”, “pháp lý rõ”).

4. **Chunk (data_pipeline/chunk.py):** Chia mỗi listing thành 4 loại chunk ngữ nghĩa độc lập:
 - `overview` – Tổng quan: tiêu đề, giá, diện tích, khu vực, pháp lý, nội thất.
 - `description` – Mô tả chi tiết từ người đăng.
 - `location` – Vị trí: địa chỉ, quận/huyện, tỉnh/thành phố.
 - `intent_tags` – Nhu cầu phù hợp (dựa trên intent tagging).
5. **Embed (data_pipeline/embed.py):** Tạo vector embedding 1024 chiều cho từng chunk bằng BGE-M3 (BAAI/bge-m3). Sử dụng sentence-transformers với `batch_size=16`, `normalize=True`, retry 3 lần exponential backoff.
6. **Load DB (data_pipeline/load_db.py + ingestors/):** Upsert vào PostgreSQL qua asyncpg driver với batch processing (25–50 bản ghi/batch). Dữ liệu quan hệ → bảng listings; dữ liệu vector → bảng chunks với HNSW index (`m=16`, `ef_construction=64`).
7. **Serve (FastAPI + Next.js):** Backend API truy xuất dữ liệu qua hybrid search (SQL filtering + pgvector kNN + Cohere rerank + Redis cache). Frontend hiển thị qua Next.js App Router với SSR.
8. **Schedule (Apache Airflow):** Toàn bộ pipeline từ bước 1–6 được lập lịch tự động qua 4 DAGs (daily, weekly×2, monthly) với retry policy và alerting.

Luồng xử lý Chatbot

1. **User gửi query** qua ChatWidget (Frontend).
2. **Frontend gọi** `POST /api/v1/chat` (Backend).
3. **Backend xác thực** (JWT hoặc anonymous), kiểm tra quota, lưu ChatMessage.
4. **Router phân loại ý định** (keyword-based hoặc Gemini-based).
5. **Orchestrator chọn agents** và gọi song song qua `asyncio.gather()`.
6. **Agent thực thi** hybrid search (SQL + pgvector kNN ± Cohere rerank ± Redis cache).
7. **Kết quả tổng hợp** → AgentChatResponse (`final_response`, `sources`, `suggested_actions`).

8. **Nếu Agent Service được kích hoạt**, Backend ủy thác sang Agent Service (LangGraph 8-node workflow) thay vì pipeline nội bộ.
9. **Backend lưu** ChatMessage, AgentTrace và trả kết quả về Frontend.
10. **Frontend hiển thị** phản hồi kèm sources và suggested actions.

Phụ lục C DANH SÁCH API CHÍNH

Bảng 3.3 liệt kê các API endpoint chính của hệ thống (33 endpoints, 7 routers + metrics). Tất cả endpoint đều có tiền tố `/api/v1` trừ `/metrics` và `/api/v1/health`.

Endpoint	Method	Mô tả
Listings		
GET <code>/listings</code>	GET	Danh sách BĐS (phân trang, 12+ bộ lọc: search, type, city, district, price, area, bedrooms,...)
GET <code>/listings/{id}</code>	GET	Chi tiết một BĐS (30+ trường)
GET <code>/listings/by-product-id/{pid}</code>	GET	Tìm BĐS theo product_id gốc từ nguồn crawl
GET <code>/listings/similar/{id}</code>	GET	BDS tương tự (cùng quận, loại hình, phân khúc giá)
Market		
GET <code>/market/stats</code>	GET	Tổng quan thị trường (tổng tin, giá TB, diện tích TB, số TP/quận)
GET <code>/market/top-locations</code>	GET	Top khu vực nhiều tin nhất (lọc theo listing_type)
GET <code>/market/price-by-district</code>	GET	Thống kê giá theo quận (avg, min, max, giá/m ²)
GET <code>/market/property-types</code>	GET	Phân bố số lượng theo loại hình BĐS
GET <code>/market/cities</code>	GET	Danh sách thành phố + số lượng tin
GET <code>/market/districts</code>	GET	Danh sách quận/huyện (có thể lọc theo city)
GET <code>/market/categories</code>	GET	Danh sách tất cả loại hình BĐS
Chat		
POST <code>/chat</code>	POST	Gửi tin nhắn đến chatbot (tự động tạo session nếu chưa có)
GET <code>/chat/sessions</code>	GET	Danh sách phiên chat của user (cần xác thực)
GET <code>/chat/sessions/{id}</code>	GET	Lịch sử tin nhắn của một phiên chat
POST <code>/chat/feedback</code>	POST	Gửi đánh giá (rating, issue_type, comment)
Auth		
POST <code>/auth/register</code>	POST	Đăng ký tài khoản (email, password, full_name)
POST <code>/auth/login</code>	POST	Đăng nhập, nhận JWT token (hết hạn sau 24h)
GET <code>/auth/me</code>	GET	Thông tin người dùng hiện tại (cần xác thực)
Preferences & Memory		
GET/PATCH <code>/preferences</code>	GET/PATCH	Xem/cập nhật sở thích người dùng (cần xác thực)
POST <code>/memory-proposals/{id}/accept</code>	POST	Chấp nhận đề xuất ghi nhớ của chatbot
POST <code>/memory-proposals/{id}/reject</code>	POST	Từ chối đề xuất ghi nhớ
Admin (cần quyền admin)		
GET <code>/admin/chat-traces</code>	GET	Danh sách agent traces (intent, agents, latency)
GET <code>/admin/chat-traces/{id}</code>	GET	Chi tiết một trace (từng bước, input/output)
GET <code>/admin/pipeline-readiness</code>	GET	Trạng thái sẵn sàng của các nguồn dữ liệu

GET /admin/eval-runs	GET	Danh sách kết quả đánh giá LLM Judge
GET /admin/feedback	GET	Danh sách phản hồi người dùng về chatbot
GET /admin/memory-proposals	GET	Danh sách memory proposals đang chờ xử lý
GET /admin/agent-health	GET	Thống kê sức khỏe agent theo trạng thái
GET /admin/top-queries	GET	Top truy vấn phổ biến (đang phát triển)
System		
GET /api/v1/health	GET	Health check endpoint
GET /metrics	GET	Prometheus metrics (chat requests, latency, pipeline)

Bảng 3.3: Danh sách API endpoints chính của hệ thống (tiền tố mặc định: /api/v1)

Phụ lục D VÍ DỤ PHẢN HỒI CHATBOT

Ví dụ 1: Tìm kiếm bất động sản

User: “Tôi muốn mua căn hộ chung cư ở Quận 7, giá dưới 5 tỷ, 2 phòng ngủ”

Intent: property_search

Agent sử dụng: property_search

Bộ lọc trích xuất: {listing_type: "sale", property_type: "apartment", district: "Quận 7", max_price: 5, bedrooms: 2}

Phản hồi: Dựa trên yêu cầu của bạn, tôi tìm thấy một số căn hộ phù hợp tại Quận 7:

1. **Căn hộ chung cư ABC** – 4,2 tỷ, 72m², 2PN, 2WC, Phường Tân Phong. View sông, nội thất đầy đủ, pháp lý sổ hồng.
2. **Căn hộ Sunshine City** – 4,8 tỷ, 78m², 2PN, 1WC, Phường Phú Thuận. Gần trường học, an ninh tốt, có hồ bơi.
3. **Căn hộ Park View** – 3,9 tỷ, 68m², 2PN, 1WC, Phường Tân Quy. Giá tốt nhất khu vực, gần siêu thị.

Nguồn tham khảo: [Link BĐS 1] (độ liên quan: 0.95), [Link BĐS 2] (0.89), [Link BĐS 3] (0.82)

Gợi ý tiếp theo:

- “So sánh giá/m² các căn hộ này”
- “Thủ tục pháp lý khi mua chung cư cần những gì?”
- “Khu vực Quận 7 có tiềm năng tăng giá không?”

Ví dụ 2: Tư vấn pháp lý + thị trường

User: “Thủ tục sang tên sổ đỏ mất bao lâu? Với lại giá đất ở Quận 9 hiện nay thế nào?”

Intent: mixed (legal_advice + market_analysis)

Agent sử dụng: legal_advisor, market_analysis

Phản hồi: Tôi sẽ giải đáp từng phần câu hỏi của bạn:

1. Thủ tục sang tên sổ đỏ:

Theo quy định hiện hành, thủ tục sang tên sổ đỏ (chuyển nhượng quyền sử dụng đất) gồm các bước chính:

- Công chứng hợp đồng chuyển nhượng tại văn phòng công chứng (1–2 ngày).
- Khai thuế thu nhập cá nhân và lệ phí trước bạ tại Chi cục Thuế (3–5 ngày).
- Nộp hồ sơ sang tên tại Văn phòng Đăng ký Đất đai (10–15 ngày làm việc).

Tổng thời gian ước tính: **3–5 tuần** tùy địa phương.

Lưu ý quan trọng: Cần kiểm tra đất không có tranh chấp, không bị quy hoạch, và đã hoàn thành nghĩa vụ tài chính.

2. Giá đất tại Quận 9:

Hiện tại (dữ liệu cập nhật từ hệ thống):

- Giá đất nền trung bình: **35–55 triệu/m²** tùy vị trí.
- Giá đất mặt tiền đường lớn: **60–90 triệu/m²**.
- Xu hướng: tăng nhẹ 5–8% so với cùng kỳ năm trước, được thúc đẩy bởi hạ tầng giao thông (Metro, Vành đai 3).

Nguồn tham khảo: [Link KB Pháp lý 1], [Link KB Pháp lý 2], [Link Thống kê Quận 9]

Gợi ý tiếp theo:

- “Các loại thuế phí khi mua đất là gì?”
- “So sánh giá đất Quận 9 với Quận 2 và Thủ Đức”
- “Có nên đầu tư đất nền Quận 9 thời điểm này?”

CHỈ MỤC

TÀI LIỆU THAM KHẢO